studocu

Advanced Operating System

M.sc(Computer Science) (Thiruvalluvar University)

Scan to open on Studocu

Studocu is not sponsored or endorsed by any college or university
Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

SHANMUGA INDUSTRIES ARTS & SCIENCE COLLEGE,T.V.MALAI.

PG AND RESEARCH DEPARTMENT OF COMPUTER SCIENCE

CLASS: I MSC. COMPUTER SCIENCE

SUBJECT CODE:

SUBJECT NAME: ADVANCED OPERATING SYSTEM

SYLLABUS

Unit-I

BASICS OF OPERATING SYSTEMS

Basics of Operating System: What is an operating systems? -Main frame systmes -Desktop systems-Multiprocessor systems-Distributed systems-Clustered Systems-Real Time Systems-Handled Systems-Feature Migration –Computing Environments-Process Scheduling-Cooperating Process –Interprocess Communicationb-Deadlocks-Prevention-Avoidence-Detection-Recovery.

Operating system

What is an operating system?

DEFINITION

"An operating system is a program that manages a computer′ s hardware. It also Provides a basis for application programs and acts as an intermediary between the Computer user and the computer hardware″ .

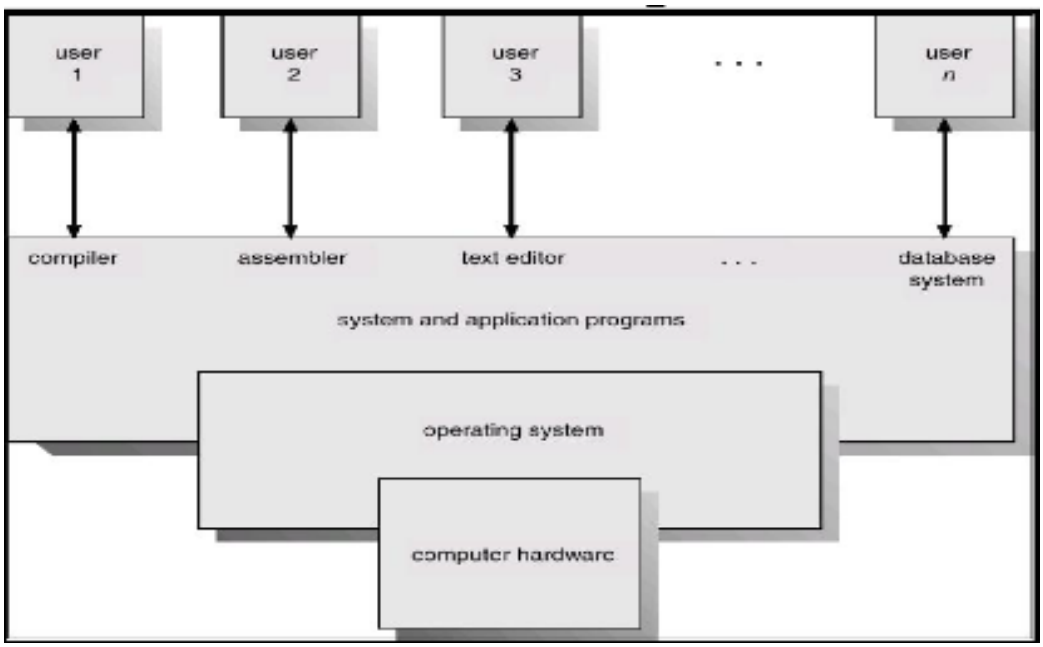
WORKING OF OPERATING SYSTEM :

A computer system can be divided into four components with hardware, The operating system, the application programs, and the user.

Figure 1.1 − Abstract View of the Components of a Computer System The hardware Central Processing Unit (CPU), the Memory, and the Input /Output (I/O) devices provides the basic computing resources for the system.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)



The Application Programs such as word processors, spreadsheets, Compilers, and Web browsers define the ways in which these resources are Used to solve users′ computing problems. A computer system as consisting of Hardware, software, and data. The operating system provides the means for Proper use of these resources in the operation of the computer system.

Operating System – Functions

To brief, Following are some of important functions of an operating System which we will look in more detail in upcoming chapters:

***Process Management**

***I/O Device Management**

***File Management**

***Network Management**

***Main Memory Management**

***Secondary Storage Management**

***Security Management**

***Command Interpreter System**

***Control over system performance**

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

***Job Accounting**

***Error Detection and Correction**

***Coordination between other software and users**

***Many more other important tasks**

Advantages of Operating System

- **Computing Source**
- **Resource Sharing**
- **User-Friendly Interface**
- **No Coding Lines**
- **Data Protection**
- **Software Update**
- **Multitasking**

Disadvantages of the Operating System

- **System Failure**
- **Expensive**
- **Virus Threat**
- **Reliability**

What is a Mainframe Computer?

Mainframes are a type of computers, which are made for ′throughput′ as fast as possible; a throughput can be defined as ″the rate at which the data is processed″ . And also mainframes are majorly used for transaction processing; a transaction can be defined as ″a set of operations including disk read and write, operating system calls, transferring data from one subsystem to another, etc….

The mainframes have more processing power compared to servers and microcomputers (like- laptop, PC, etc…), but have less processing power compared to a supercomputer.

The main focus of the Main-frames is throughput, ″A throughput is a rate at which something is processed.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

Components of a Mainframe Computer

The mainframes have two types of processors, the first one is the Main processor and another one is System assistance Processor or SAP. The SAP processors do not process any data but move the data from one place to another as fast as possible.

Each processor can have up to 7 to 10 cores which are specially designed and engineered for ′higher throughput′ .

Each mainframe can have up to 160 of I/O cards. And also they have got some serious amount of ROM (Solid State Drives) for faster data storage and transfer.

The fact that the main-frames have so many I/O cards is because these are made for redundancy that is, if one card fails, other cards will take over the work-load of that card until the card is replaced.

Why are mainframe computers used?

Main-frames are used for Reliability, Redundancy, and availability. These are must

computers to have where ′0′ downtime is acceptable.

Due to the fact that these computers are reliable and have redundancy that is if an I/O card fails due to any reason, its workload will automatically get transferred to other I/O card ensuring ′0′ downtime

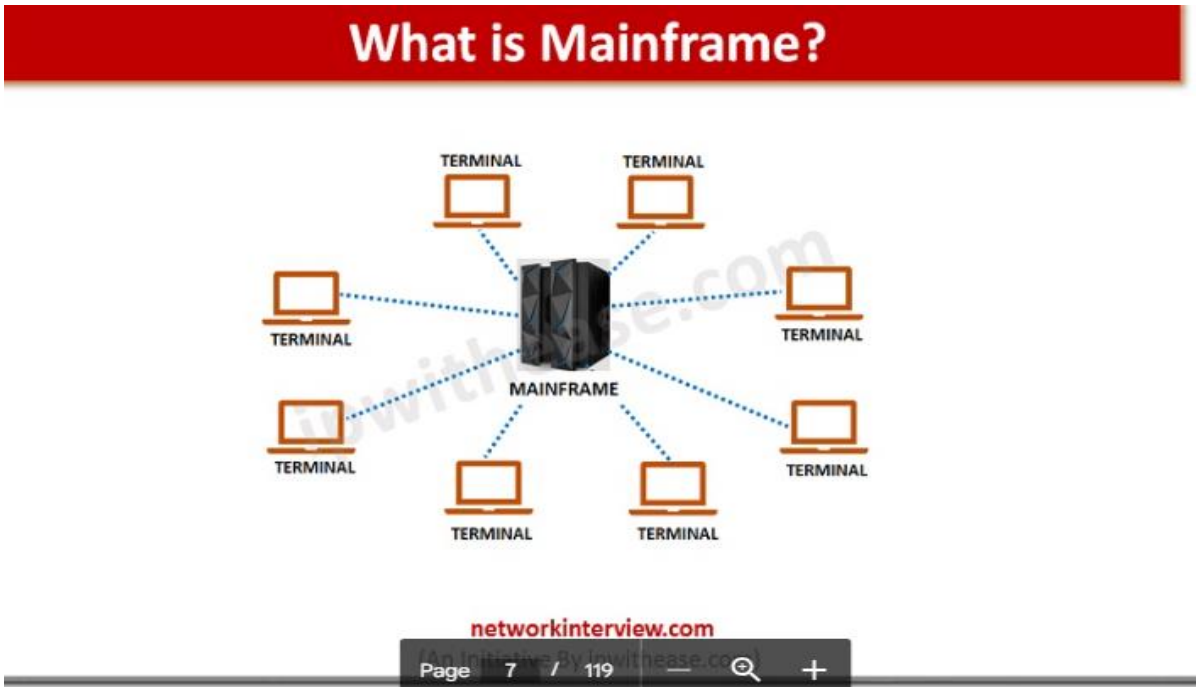
Advantages

- **The processing speed is very fast.**
- **Can handle multiple inputs at same time.**
- **Redundancy, Can withstand failure of a part without affecting the function of rest of the computer.**
- **Always available, as once started they will remain on for rest of the time.**
- **Reliability.**

Disadvantages

- **Mainframes cannot be used as a normal computer, because they are made for specific task only.**
- **It requires a special OS to run.**
- **Are very expensive.**
- **Mainly used for commercial purposes like transaction processing.**
- **Cannot calculate or solve complex problems like a supercomputer.**

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)



What is a Desktop Operating System?

Desktop OS is the operating system that is present in all personal computers. It is also known as client operating system. MS-DOS is the first discovered desktop OS. Desktop OS are written in C, C++ and assembly languages. Some OS are pre-installed while others are installed later. This software can be available for free or paid services are also available for some of them. The popular Desktop OS are Windows, Linux and Mac.

Most of these OS use Graphical User Interface (GUI) that allows the users to click on icons and open applications using pointer devices. These OS can do many tasks. They can run multiple programs at the same time. Some of the functions of desktop OS include memory management, file management, storage, managing hardware devices, running various software applications, task management etc.,

More space is required for these OS. They support multiple applications. These OS take more time to reboot and to run a process.

Desktop operating systems can also be used in super computers that function as servers. The OS that is used in these servers are Linux, Unix etc.,

Advantages

- **User friendly GUI**
- **Multi-tasking**
- **Frequent updates to keep up with the latest features**
- **Allows users to share data**

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

Disadvantages

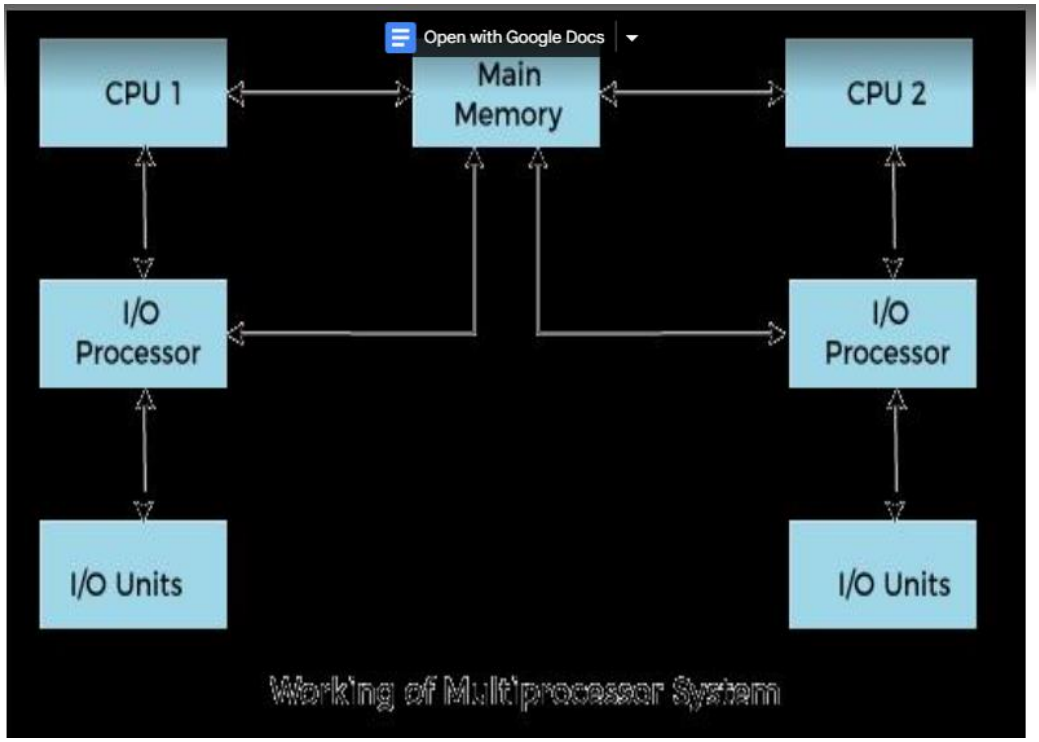
- **Some OS costs more**
- **They are more complex and difficult to understand**
- **If central OS fails, then the whole system stops working**

Multiprocessing Operating System

In multiprocessing, Parallel computing is achieved. There are more than one processors

Present in the system which can execute more than one process at the same time. This will

Increase the throughput of the system.



Advantages of Multiprocessing operating system:

● Increased reliability: Due to the multiprocessing system, processing tasks can beDistributed among several processors. This increases reliability as if one processor Fails, the task can be given to another processor for completion.

● Increased throughput: As several processors increase, more work can be done in Less.

Disadvantages of Multiprocessing operating System

● Multiprocessing operating system is more complex and sophisticated as it takes care Of multiple CPUs simultaneously.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

SHANMUGA INDUSTRIES ARTS & SCIENCE COLLEGE,T.V.MALAI

PG &RESEARCH DEPARTMEMT OF COMPUTER SCIENCE

CLASS : I MSc Computer Science

SUBJECT CODE :

SUBJECT NAME : Advanced Operating System

UNIT II

DISTRIBUTED OPERATING SYSTEMS

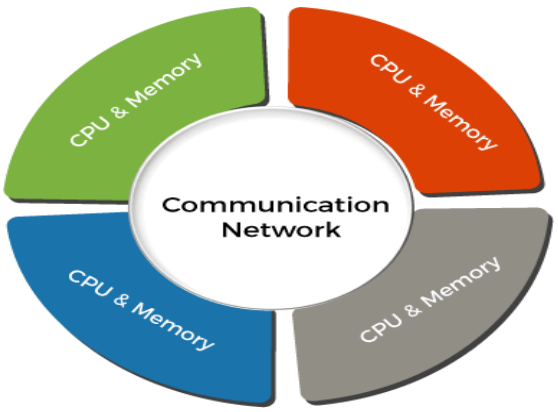
Distributed Operating Systems: Issues – Communication Primitives – Lampert's Logical Clocks – Deadlock handling strategies – Issues in deadlock detection and resolution- distributed file systems –design issues – Case studies – The Sun Network File System- Coda.

Distributed Operating System

A distributed operating system **(DOS)** is an essential type of operating system.

Distributed systems use many central processors to serve multiple real-time applications and users. As a result, data processing jobs are distributed between the processors.

It connects multiple computers via a single communication channel. Each of these systems has its own processor and memory. Additionally, these **CPUs** communicate via high-speed buses or telephone lines.



This operating system consists of numerous computers, nodes, and sites joined together via **LAN/WAN** lines. It enables the distribution of full systems on a couple of center processors, and it supports many real-time products and different users. Distributed operating systems can share their computing

Downloaded by bhagavathi 9411 (bhagavathi94112008@gmail.com)

resources and I/O files while providing users with virtual machineabstraction.

Types of Distributed Operating System

There are various types of Distributed Operating systems. Some ofthem are as follows:

1. **Client-Server Systems**
2. **Peer-to-Peer Systems**
3. **Middleware**
4. **Three-tier**
5. **N-tier**

Client-Server System

- This type of system requires the client to request a resource, after which the server gives the requested resource. When a client connects to a server, the server may serve multiple clients at the same time.
- Client-Server Systems are also referred to as "Tightly Coupled Operating Systems". This system is primarily intended for multiprocessors and homogenous multicomputer.

Server systems can be divided into two parts:

1. Computer Server System

This system allows the interface, and the client then sends its own requests to be executed as an action. After completing the activity,it sends a back response and transfers the result to the client.

2. File Server System

It provides a file system interface for clients, allowing them to execute actions like file creation, updating, deletion, and more.

Peer-to-Peer System

- The nodes play an important role in this system. The task is evenly distributed among the nodes. Additionally, these nodes can share data and resources as needed. Once again, they require a network to connect.
- The Peer-to-Peer System is known as a "Loosely Couple System". This concept is used in computer network applications since they contain a large number of processors that do not share memory or clocks.
- Each processor has its own local memory, and they interact with one another via a variety of communication methods liketelephone lines or high-speed buses.

Middleware

Middleware enables the interoperability of all applications runningon different operating systems. Those programs are capable oftransferring all data to one other by using these services.

This document is available on studocu

Downloaded by bhagyashri 9411 (bhagyashri94112008@gmail.com)

Three-tier

The information about the client is saved in the intermediate tier rather than in the client, which simplifies development. This type of architecture is most commonly used in online applications.

N-tier

When a server or application has to transmit requests to other enterprise services on the network, n-tier systems are used.

Features of Distributed Operating System

There are various features of the distributed operating system. Some of them are as follows:

Openness

It means that the system's services are freely displayed through interfaces. Furthermore, these interfaces only give the service syntax. For example, the type of function, its return type, parameters, and so on. Interface Definition Languages are used to create these interfaces (IDL).

Scalability

It refers to the fact that the system's efficiency should not vary as new nodes are added to the system. Furthermore, the performanceof a system with 100 nodes should be the same as that of a system with 1000 nodes.

Resource Sharing

Its most essential feature is that it allows users to share resources. They can also share resources in a secure and controlled manner. Printers, files, data, storage, web pages, etc., are examples of shared resources.

Flexibility

A DOS's flexibility is enhanced by modular qualities and delivers amore advanced range of high-level services. The kernel/ microkernel's quality and completeness simplify the implementation of such services.

Transparency

It is the most important feature of the distributed operating system. The primary purpose of a distributed operating system is to hide the fact that resources are shared. Transparency also implies that the user should be unaware that the resources he is accessing are shared.

Heterogeneity

The components of distributed systems may differ and vary in operating systems, networks, programming languages, computer hardware, and implementations by different developers.

Downloaded by bhagyashri 9411 (bhagyashri94112008@gmail.com)

Fault Tolerance

Fault tolerance is that process in which user may continue their work if the software or hardware fails.

Examples of Distributed Operating System

✓ **Solaris:**

It is designed for the SUN multiprocessor workstations

✓ **OSF/1:**

It's compatible with Unix and was designed by the OpenFoundation Software Company.

✓ **Micros:**

The MICROS operating system ensures a balanceddata load while allocating jobs to all nodes in the system.

✓ **DYNIX:**

It is developed for the Symmetry multiprocessor computers.

✓ **Locus:**

It may be accessed local and remote files at the same time without any location hindrance.

✓ **Mach:**

It allows the multithreading and multitasking features.

Applications of Distributed Operating System

➢ **Network Applications:**

DOS is used by many network applications, including the Web, peer-to-peer networks, multiplayer web- based games, and virtual communities.

➢ **Telecommunication Networks:**

DOS is useful in phones and cellular networks. A DOS can be found in networks like the Internet, wirelesssensor networks, and routing algorithms.

➢ **Parallel Computation:**

DOS is the basis of systematic computing, which includes cluster computing and grid computing, and a varietyof volunteer computing projects.

➢ **Real-Time Process Control:**

This document is available on studocu

Downloaded by bhagyashri 9411 (bhagyashri94112008@gmail.com)

real-time process control system operates with a deadline, and such examples include aircraft control systems.

Advantages and Disadvantages of Distributed Operating System.

Advantages:

1. It may share all resources (CPU, disk, network interface, nodes, computers, and so on) from one site to another, increasing data availability across the entire system.
2. It reduces the probability of data corruption because all data is replicated across all sites; if one site fails, the user can access data from another operational site.
3. The entire system operates independently of one another, andas a result, if one site crashes, the entire system does not halt.
4. It increases the speed of data exchange from one site to another site.
5. It is an open system since it may be accessed from both local and remote locations.
6. It helps in the reduction of data processing time.

Disadvantages

1. It is hard to implement adequate security in DOS since the nodes and connections must be secured.
2. The database connected to a DOS is relatively complicated and hard to manage in contrast to a single-user system.
3. The underlying software is extremely complex and is notunderstood very well compared to other systems.
4. The more widely distributed a system is, the more communication latency can be expected. As a result, teams and developers must choose between availability, consistency, and latency.
5. Gathering, processing, presenting, and monitoring hardware use metrics for big clusters can be a real issue.

ISSUES in DS:

Causes of Operating System Failure:

✓ A system failure may occur due to a hardware failure or a significant software problem, leading the system to freeze, reboot, or stop working completely. An error may or may notbe displayed on the screen because of a system failure.

✓ The computer may shut down without warning or error message. If an error message is presented on Windows PCs, it is frequently displayed as a Blue Screen of Death error.

There are mainly two reasons behind the operating system failure.These reasons are as follows:

1. Software Problems

Downloaded by bhagyashri 9411 (bhagyashri94112008@gmail.com)

2. Hardware Problems

Software Problems:

There are various software problems that cause the operatingsystem failure. Some software problems are as follows:

1. Improper Drivers

- You need drivers to use additional hardware, which can typically be downloaded from the internet. These drivers could be infected with bugs. These flaws cause the operating system to crash.
- Most modern operating systems include the **"Safe Mode Boot"** option. Safe Mode Boot is used for troubleshooting and locating faulty drivers. Only the most critical drivers are loaded in Safe Mode Boot, not all of them.

2. Thrashing

- Deadlock happens when two programs are running need control over a particular resource. The OS may attempt to switch back and forth between the two programs during a deadlock'

- It eventually leads to Thrashing, in which the hard disk is overworked by excessively shifting information between system memory and virtual memory, causing a system crash.

3. Corrupt Registry

4. The registry is a small database that stores all of the detail about the kernel, drivers, and programs. The OS searches its registry before beginning any app.

- Registry corruption may occur as a result of erroneous application removal, careless registry changes, or having too many installed applications, among other things.

5. Virus

- On the system, a virus may replicate itself. Viruses are particularly dangerous since they can modify and delete user files and cause machines to crash.

- A virus is a small piece of code that is embedded in system software. The virus becomes embedded in other files and programs as the user interacts with the program, potentially rendering the system unworkable.

6. Trojan Horse

The application saves the user's login details. It prevents userdetails from being transferred to a rogue user, who can subsequently log in and access system resources.

7. Slow System Performance

- The system's performance has become very slow. If you're looking for how to recognize signs of operating system failureon the internet, it is the ideal sign.

- you can check to see if you have installed the latest versions of Windows on the

This document is available on studocu

Downloaded by bhagyashri 9411 (bhagyashri94112000@gmail.com)

system. Even security fixes must be kept up to date. After that, the system will resume normal operation.

8. Failure to Boot

- If you are unable to boot, the OS may have been damaged. There have been changes to the system's boot order. You can examine the booting process and sequence setup.

- In the case of an OS failure, you must reinstall the Windows OS. Please keep in mind that the problem might be significant. It is one of the best indicators of operating systemfailure.

9. Compatibility Error

- This type of issue commonly arises when the old apps in Windows no longer work due to the upgrade. When you encounter this issue, you are aware that it is one of the operating system breakdown symptoms, but you can easily address it.

- In most cases, Windows offers an in-built capability that allows applications to be made compatible with the new version. If you are a computer expert and are familiar with the language, you can run the software in compatibility mode.

Hardware Problems:

There are various hardware problems that cause the operating system failure. Some hardware problems are as follows:

1. Power Problem

The System Power Supply's improper operation can result in the System being shut down immediately.

2. Overheating

Overheating is the important hardware issue of operating system failure. Overheating is a simple one to rule out. A fan is built intoa computer's CPU to keep it cool. The fan may get worn andinefficient over time, or it may just be unable to handle your computer's job.

3. Motherboard Failure

A failed motherboard might cause a system failure as the computer is unable to process requests or function in general.

4. RAM

A faulty RAM chip might cause system failures as the OS is unable to access data stored on the RAM chip.

5. Bad Processor

A faulty processor can and typically causes a system failure as thesystem may not function if the CPU is not functioning properly.

Comminicative primitive:

Downloaded by bhagyashri 9411 (bhagyashri94112008@gmail.com)

Models For Distributed Applications:

There are two models that are widely accepted to develop distributed operating system.

1. Message Passing
2. Remote procedure call.

1. Message Passing Model:

1. The Message Passing model provides two basiccommunication Primitives

2. The send Primitives has two parameters:

- A message and its destination.

3. The send primitive has also two parameters:

- The source of a message and a buffer for storingthe message.

4. An application of these primitives can be found in clientserver computation model.

2. Blocking VS. Non Blocking Primitives:

1. In the standard message passing model messages are copiedthree times

- From user buffer to kernel buffer
- From kernel buffer on sending computer to kernelbuffer to the kernel buffer on receiving computer.
- From receiving buffer to user buffer.

2. With non-blocking primitive, the send primitive return thecontrol to user process.

3. While the Receiving primitive respond by signaling and provide abuffer to copy.

4.A significant disadvantages of non-blocking is that programmingbecomes difficult.

5. In the unbuffered option, data is copied from one user buffer to another user directly.

6. With Blocking primitives, the send primitive does not return the control to the user program until the message has been sent (an unreliable blocking primitive) or until an acknowledgment hasbeen received (a reliable blocking primitive).

7. In both cases user buffer can be reused. <u>Synchronous Vs AsynchronousPrimitives:</u>

- With synchronous primitive, a send primitive is block until a corresponding Receive primitive is executed at the receiving computer.

- With asynchronous primitive,the messages are buffered.•A send primitive is not block even if nocorresponding executionof a receive primitive.

This document is available on studocu

Downloaded by bhagyashri 9411 (bhagyashri94112008@gmail.com)

Remote Procedural Call:

1.A More natural way to communicate is throughProcedural call:

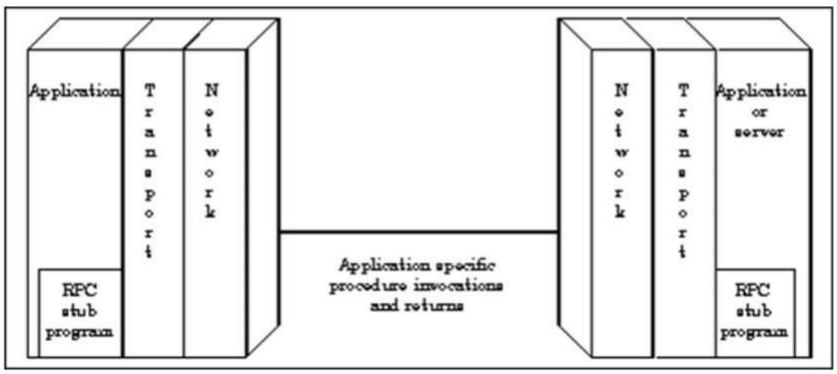
- every language supports it.
- semantics are well defined and understood.
- natural for programmers to use.

2. Programmer Using such a model must handle the following details:

- Pairing of responses with request messages.
- Data representation.
- Knowing the address of remote machine on theserver
- Taking care of communication and system failure

Downloaded by bhagyashri 9411 (bhagyashri9411@gmail.com)

RPC



Basic RPC Operation:

1. The RPC Mechanism is based on the observation that a procedural call is well known for transfer of control and data with in a program running an a single machine.
2. On invoking a remote procedure, the calling process is suspended.•If any parameter are passed to the remote machine where the procedure will execute.

3. On completion, the result are passed back from server to client and resuming execution as if it had called a localprocedure.

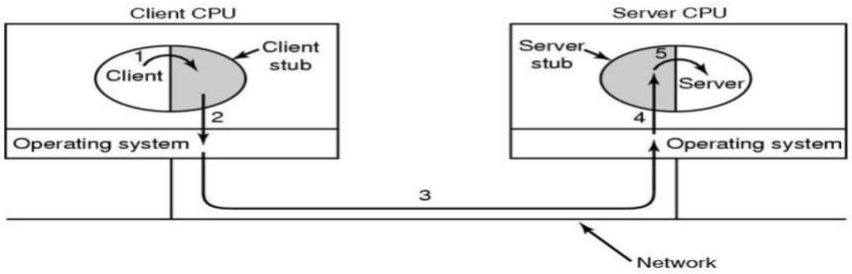
Design Issues in RPC:

- RPC mechanism is based on the concept of stub procedures.

- The server writer writes the server and links it withthe server-side stubs; the client writes her programand links it with the client-side stub.

- The stubs are responsible for managing all details of the remote communication between client and server.

Design Issues (Contd)



Structure:

- When a program (client) makes a remote procedure call, say p(x,y), it

This document is available on studocu
Downloaded by bhagyashri 9411 (bhagyashri94112008@gmail.com)

actually makes a local call on a dummy procedure or a client-side stub procedure p.

- The client-side stub procedure contruct a message containing the identity of the remote procedure andparameters and then send to remote machine.

- A stub procedure at server side stub receives the message and makes a local call to the procedure specified in the message.

- After execution control returns to the server stub procedure which return the control to client side stub.The stub procedures can be generated at compile time or can be linked at run time.

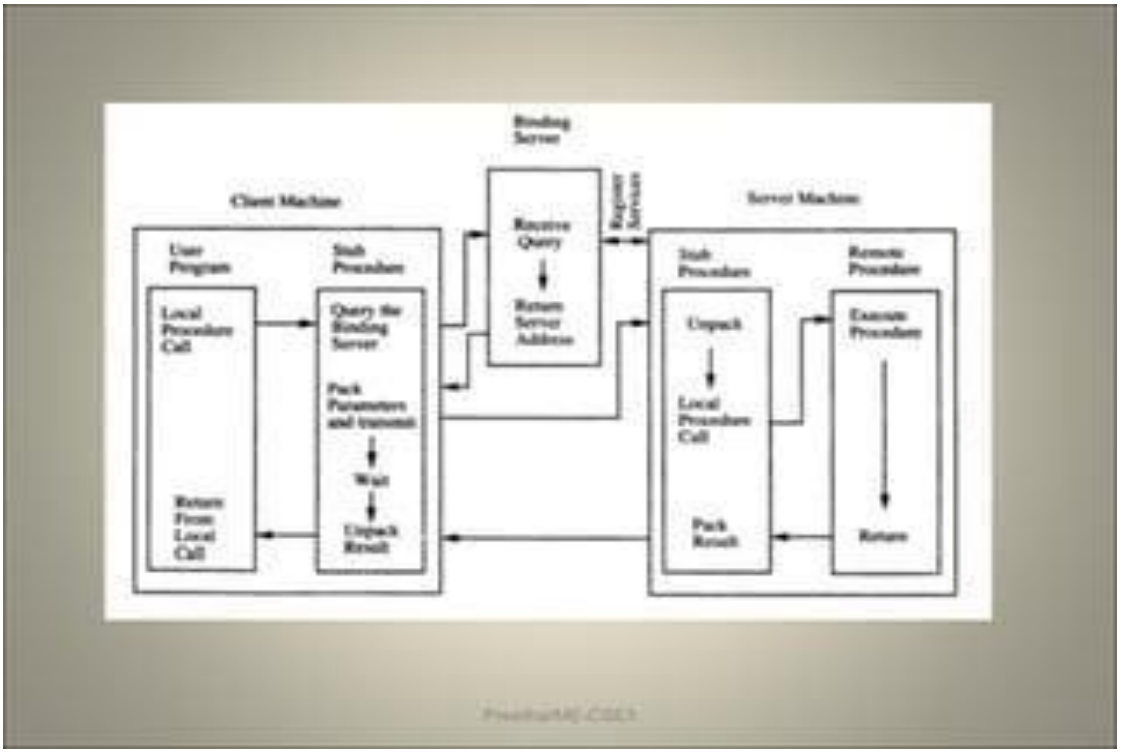
Binding:

- Binding is process that determines the remoteprocedure, and the machine on which it will beexecuted.

- It may also check the compatibility of parameters passed and procedure type called.

- Binding server essentially store the server machine along with the services they provide.

- Another approach used for binding is where the client specifies the machine and the servicerequired and the binding server returns the port number for communication.



RPC Problems:

1. Procedures reside on different machines

- This means we cannot simply jump to the start of theprocedure

Downloaded by bhagyashri9411 (bhagyashri9411@gmail.com)

- We need to use network communication techniques tointeract with the remote machine.

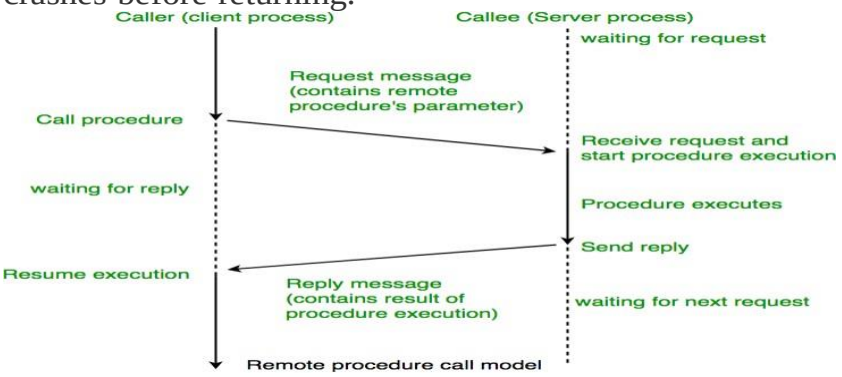
2. Procedures reside in different address spaces

- This means that we cannot pass pointers from caller tocallee because a pointer is only valid in one AS

3. Parameters and results need to be passed across te network4.Machine can

crash

- what happens if you call a remote procedure and theremote machine crashes before returning.



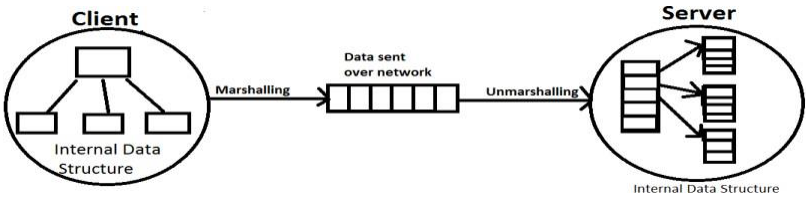
Marshalling:

- RPC uses message passing paradigm to communicate over the communication network

- The client and server stubs are required to pack and unpack parameters to form messages

- This task is known as parameter marshallingMarshalling is a relatively

complex activity·



- The server may implement multiple procedures and it may be necessary for the client to specify which procedure is to be invoke

- Different machines may use different character representation

- Machine may represent integers differentlFloating point numbers may also be different

- We must therefore send data from one machine to another in some canonical form .For example, the Sun XDR standard. Worst of all, any pointer-based data

This document is available on studocu
Downloaded by bhagyashri 9411 (bhagyashri94112000@gmail.com)

structures cannot be passed because client-side pointers will not be valid on the server

- There are two main solutions to this problem:

1. Forbid pointer-based data structures
2. Encode the dta

- When encoding the data, we need to be careful to avoid losing referential integrity

RPC Tools:

- Fortunately, we don't have to write the client and server stub (skeleton) code ourselvesInstead

- we use tools that do most of the work for us
- These tools are known as interface generators or stubgenerators

1. The most common one is RPCGEN
2. Also, Common Object Request Broker Architecture (CORBA)

- Stub generators use an interface definitionlanguage (IDL) to describe the client and server code

<u>Paradigm for Building DistributedPrograms:</u>

- There are two approaches for distributed Application.
- Communication Oriented Design

1. Begins with communication protocol
2. Message format and syntax
3. Design client sever component by specifying interactionwith incoming and outgoing messages.

- Application Oriented

1. Begin with the Application
2. Build and test a working version that operate on single machine
3. Divide the program into two or more pieces and add communication protocol to allow each piece to execute on separate machine

<u>Client/Server and RPC:</u>

- Like conventional procedure call, the system, a remote procedure call transfers control to the called procedure.

- The system suspend the execution of calling procedure during the call and only allows the called procedure to execute.

- When a remote program issues a response, it corresponds to the execution of a return in a conventional procedure call.

- Controls flows back to the caller and called procedure ceases to execute.

Downloaded by bhagyashri 9411 (bhagyashri9411@gmail.com)

- One remote procedure may call another remote procedure.

- Conventional procedures actually accept a few arguments and return only a few result.
- Server can accept or return arbitrary amount of data.

- It would be ideal if local and remote procedural callsbehaved identical

a. Network delays
b. Same address space

- Remote procedure call cannot pass pointer as arguments RPC cannot share the Caller's Environment

Middleware And Object Oriented Middleware:

- Variety of commercial tools are available that use the RPC paradigm to construct client-serversoftware.
- These tools are generally called middleware becausethey fits between a conventional application program and the network software.
- The role of middleware is to ease the task of designing, programming and managing distributed applications by providing a simple, consistent and integrated distributed programming environment.

RPC Tools (examples):

MSRPC:

1. Microsoft Remote Procedural Call
2. Derived from DCE RPC

- Own IDL and Protocol that client and server stubs uses tocommunicate

COBRA:

- Common object Request broker Architecture permits anentire object to placed at server
- More Dynamic
- In conventional RPC programmers uses tool to create stubprocedure while in Cobra software creates proxies at run time

MSRPC2:

- Microsoft develop second generation of MSRPC.COM/DCOM:

- Microsoft also develop COM and DCOM.
- The standard define a binary package referred to as packaging scheme.

✓ All object are given globally unique names and allobject references

This document is available on studocu
Downloaded by Gnanavathi Ganesan (gnanavathi0411@gmail.com)

use the global scheme.

- Distributed component object model extend com by creatingapplication level protocols.
- The Combination of COM and DCOM is called COM+.

Lamport's Logical Clock:

Lamport's Logical Clock was created by Leslie Lamport. It is a procedure to determine the order of events occurring. It provides a basis for the more advanced Vector Clock Algorithm. Due to the absence of a Global Clock in a Distributed Operating System Lamport Logical Clock is needed.

Algorithm:

- Happened before relation(->): a -> b, means 'a' happenedbefore 'b'.
- Logical Clock: The criteria for the logical clocks are:
- [C1]: Ci (a) < Ci(b), [Ci -> Logical Clock, If 'a' happened before'b', then time of 'a' will be less than 'b' in a particular process.

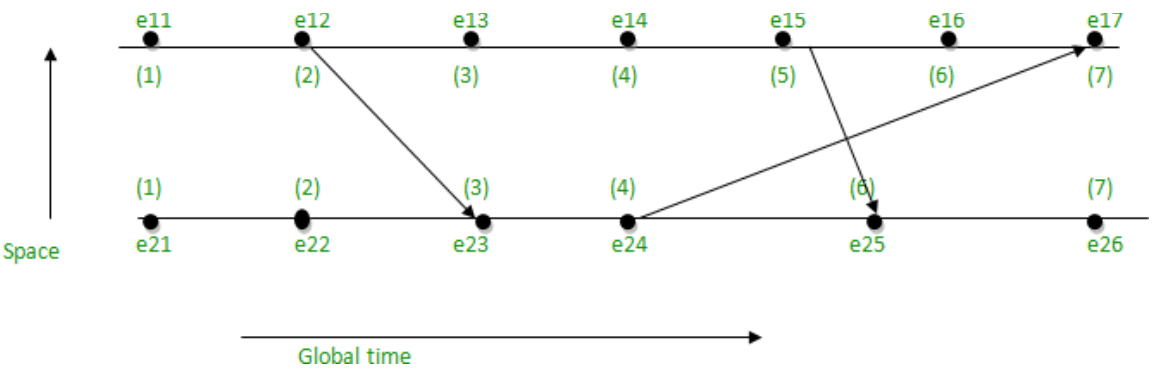
]

- [C2]: Ci(a) < Cj(b), [Clock value of Ci(a) is less than Cj(b)]Reference:

- Process: Pi
- Event: Eij, where i is the process in number and j: jth eventin the ith process.
- tm: vector time span for message m.
- Ci vector clock associated with process Pi, the jth element is Ci[j] and contains Pi's latest value for the current time in process Pj.
- d: drift time, generally d is 1. Implementation Rules[IR]:

- [IR1]: If a -> b ['a' happened before 'b' within the same process] then, Ci(b) =Ci(a) + d
- [IR2]: Cj = max(Cj, tm + d) [If there's more number of processes, then tm = value of Ci(a), Cj = max value between Cj and tm + d]

For Example:



- Take the starting value as 1, since it is the 1st event and there is no incoming value at the starting point:

Downloaded by bhagyashri9411 (bhagyashri94112008@gmail.com)

e11 = 1

e21 = 1

- The value of the next point will go on increasing by d (d = 1), if there is no incoming value i.e., to follow [IR1].

e12 = e11 + d = 1 + 1 = 2e13 = e12 + d = 2 + 1 = 3e14 = e13 + d = 3 + 1 = 4e15 = e14

+ d = 4 + 1 = 5e16 = e15 + d = 5 + 1 = 6e22 = e21 + d = 1 + 1 = 2e24 = e23 + d = 3 + 1

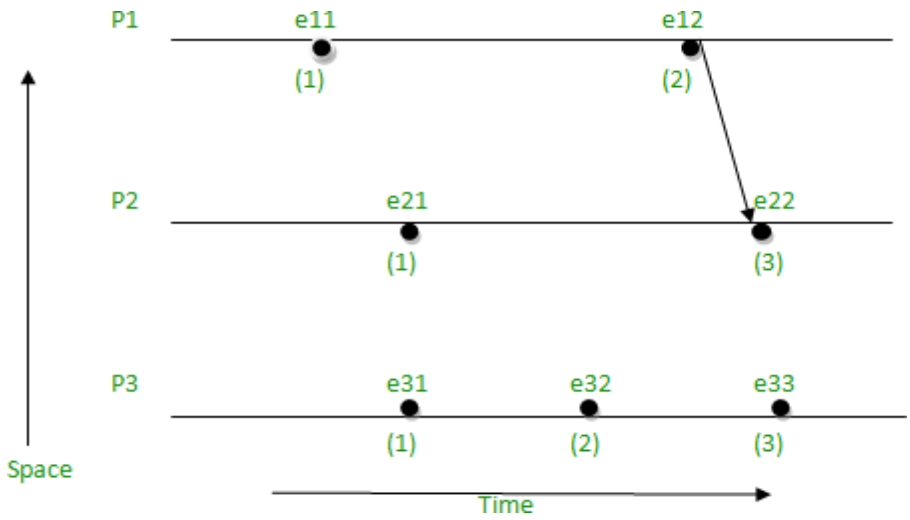
= 4e26 = e25 + d = 6 + 1 = 7

- When there will be incoming value, then follow [IR2] i.e., take the maximum value between Cj and Tm + d.

e17 = max(7, 5) = 7, [e16 + d = 6 + 1 = 7, e24 + d = 4 + 1 = 5,
maximum among 7 and 5 is 7]. e23 = max(3, 3) = 3, [e22 + d = 2 + 1 = 3, e12 + d = 2 + 1 = 3,
maximum among 3 and 3 is 3]

e25 = max(5, 6) = 6, [e24 + 1 = 4 + 1 = 5, e15 + d = 5 + 1 = 6,
maximum among 5 and 6 is 6]Limitation:

- In case of [IR1], if a -> b, then C(a) < C(b) -> true.
- In case of [IR2], if a -> b, then C(a) < C(b) -> May be true ormay not be true.



Below is the C program to implement Lamport's Logical Clock:

C++

#include <bits/stdc++.h>using namespace std;

This document is available on studocu

Downloaded by bhagyashri 9411 (bhagyashri9411@gmail.com)

```
int max1(int a, int b)

{

if (a > b)

return a;
else

return b;

}

void display(int e1, int e2,

int p1[5], int p2[3])

{

int i;

cout << "\nThe time stamps of ""events in P1:\n";

for (i = 0; i < e1; i++) { cout << p1[i] << " ";

}

cout << "\nThe time stamps of ""events in P2:\n";

for (i = 0; i < e2; i++) cout << p2[i] << " ";

}

void lamportLogicalClock(int e1, int e2,

int m[5][3])

{

int i, j, k, p1[e1], p2[e2];for (i = 0; i < e1; i++)

p1[i] = i + 1

for (i = 0; i < e2; i++)p2[i] = i + 1;

cout << "\t";

for (i = 0; i < e2; i++)

cout << "\te2" << i + 1;for (i = 0; i < e1; i++) {

cout << "\n e1" << i + 1 << "\t";for (j = 0; j < e2; j++)

cout << m[i][j] << "\t";
```

Downloaded by bhagavathi 9411 (bhagavathi9411@gmail.com)

```
}
for (i = 0; i < e1; i++) {
for (j = 0; j < e2; j++) {if (m[i][j] == 1) {
p2[j] = max1(p2[j], p1[i] + 1);for (k = j + 1; k < e2; k++)
p2[k] = p2[k - 1] + 1;
}
if (m[i][j] == -1) {
p1[i] = max1(p1[i], p2[j] + 1);for (k = i + 1; k < e1; k++)
p1[k] = p1[k - 1] + 1;
}
}
display(e1, e2, p1, p2);
}
int main()
{
int e1 = 5, e2 = 3, m[5][3];
m[0][0] = 0;
m[0][1] = 0;
m[0][2] = 0;
m[1][0] = 0;
m[1][1] = 0;
m[1][2] = 1;
m[2][0] = 0;
m[2][1] = 0;
m[2][2] = 0;
m[3][0] = 0;
m[3][1] = 0;
```

This document is available on studocu

Downloaded by bhagyashri 9411 (bhagyashri94112008@gmail.com)

```
m[3][2] = 0;

m[4][0] = 0;

m[4][1] = -1;

m[4][2] = 0;

lamportLogicalClock(e1, e2, m);return 0;

}
```

Deadlock handling strategies:

The following are the strategies used for Deadlock Handling in Distributed System:

1. Deadlock Prevention
2. Deadlock Avoidance
3. Deadlock Detection and RecoveryDeadlock Prevention

If we simulate deadlock with a table which is standing on its four legs then we can also simulate four legs with the four conditions which when occurs simultaneously, cause the deadlock.

However, if we break one of the legs of the table then the table will fall definitely. The same happens with deadlock, if we can be able to violate one of the four necessary conditions and don't let them occur together then we can prevent the deadlock.

1. Mutual Exclusion

- Mutual section from the resource point of view is the fact that a resource can never be used by more than one process simultaneously which is fair enough but that is the mainreason behind the deadlock.

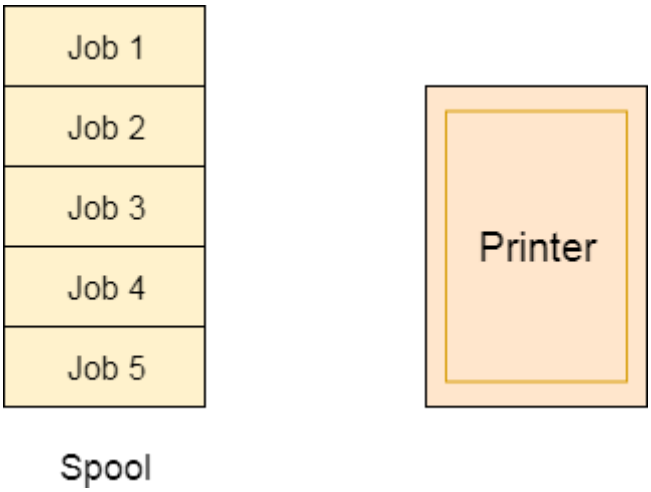
- If a resource could have been used by more than one process at the same time then the process would have never been waiting for any resource.

Spooling

- spooling can work. There is a memory associated with the printer which stores jobs from each of the process into it. Later, Printer collects all the jobs and print each one of them according to FCFS.

- By using this mechanism, the process doesn't have to wait for the printer and it can continue whatever it was doing. Later, it collects the output when it is produced.

Downloaded by bhagyashri 9411 (bhagyashri9411@gmail.com)



1. This cannot be applied to every resource.

2. After some point of time, there may arise a race condition between the processes to get space in that spool.

We cannot force a resource to be used by more than one processat the same time since it will not be fair enough and some serious problems may arise in the performance. Therefore, we cannot violate mutual exclusion for a process practically.

2. Hold and Wait

- Hold and wait condition lies when a process holds a resource and waiting for some other resource to complete its task. Deadlock occurs because there can be more than one processwhich are holding one resource and waiting for other in the cyclic order.

- However, we have to find out some mechanism by which a process either doesn't hold any resource or doesn't wait. That means, a process must be assigned all the necessary resources before the execution starts. A process must notwait for any resource once the execution has been started.

!(Hold and wait) = !hold or !wait (negation of hold and wait is, either you don't hold or you don't wait)

- This can be implemented practically if a process declares all the resources initially. However, this sounds very practical but can't be done in the computer system because a process can't determine necessary resources initially.

- Process is the set of instructions which are executed by the CPU. Each of the instruction may demand multiple resourcesat the multiple times. The need cannot be fixed by the OS.

The problem with the approach is:

1. Practically not possible.

2. Possibility of getting starved will be increases due to the fact that some process may hold a resource for a very long time.

3. No Preemption

This document is available on studocu

Downloaded by bhagyashri 9411 (bhagyashri94112008@gmail.com)

- Deadlock arises due to the fact that a process can't be stopped once it starts. However, if we take the resource away from the process which is causing deadlock then we can prevent deadlock.

- This is not a good approach at all since if we take a resource away which is being used by the process then all the work which it has done till now can become inconsistent.

- Consider a printer is being used by any process. If we take the printer away from that process and assign it to some other process then all the data which has been printed can become inconsistent and ineffective and also the fact that theprocess can't start printing again from where it has left whichcauses performance inefficiency.

4. Circular Wait

To violate circular wait, we can assign a priority number to each ofthe resource. A process can't request for a lesser priority resource. This ensures that not a single process can request a resource which is being utilized by some other process and no cycle will be formed.

Deadlock avoidance

- In deadlock avoidance, the request for any resource will be granted if the resulting state of the system doesn't causedeadlock in the system. The state of the system willcontinuously be checked for safe and unsafe states.
- In order to avoid deadlocks, the process must tell OS, the maximum number of resources a process can request to complete its execution.

- The simplest and most useful approach states that the process should declare the maximum number of resources of each type it may ever need. The Deadlock avoidance algorithm examines the resource allocations so that there can never be a circular wait condition.

Safe and Unsafe States

The resource allocation state of a system can be defined by the instances of available and allocated resources, and the maximum instance of the resources demanded by the processes.

Resources Assigned

Process	Type1	Type2	Type3	Type4
A	3	0	2	2
B	0	0	1	1

Downloaded by bhagyashri 9411 (bhagyashri94112008@gmail.com)

C	1	1	1	0
D	2	1	4	0

Resources still needed

Process	Type1	Type2	Type3	Type4
A	1	1	0	0
B	0	1	1	2
C	1	2	1	0
D	2	1	1	2

1. E = (7 6 8 4)
2. P = (6 2 8 3)
3. A = (1 4 0 1)

- Above tables and vector E, P and A describes the resource allocation state of a system. There are 4 processes and 4 types of the resources in a system. Table 1 shows the instances of each resource assigned to each process.
- Table 2 shows the instances of the resources, each process still needs. Vector E is the representation of total instancesof each resource in the system.
- Vector P represents the instances of resources that have been assigned to processes. Vector A represents the number of resources that are not in use.
- A state of the system is called safe if the system can allocate all the resources requested by all the processes without entering into deadlock.
- If the system cannot fulfill the request of all processes then the state of the system is called unsafe.
- The key of Deadlock avoidance approach is when the request is made for resources then the request must only be approvedin the case if the resulting state is also a safe state.

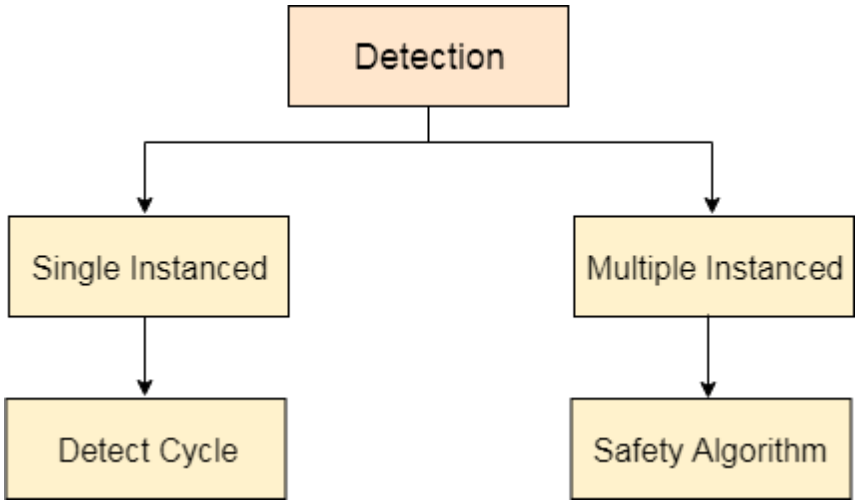
Deadlock Detection and Recovery

The OS doesn't apply any mechanism to avoid or prevent the deadlocks. Therefore the system considers that the deadlock will definitely occur. In order to get rid of deadlocks, The OS periodically checks the system for any deadlock. In case, it finds any of the deadlock then the OS will recover the system using somerecovery techniques.

This document is available on studocu

Downloaded by bhagyashri 9411 (bhagyashri94112000@gmail.com)

The main task of the OS is detecting the deadlocks. The OS can detect the deadlocks with the help of Resource allocation graph.



In single instanced resource types, if a cycle is being formed in the system then there will definitely be a deadlock. On the other hand, in multiple instanced resource type graph, detecting a cycle is not just enough. We have to apply the safety algorithm on the system by converting the resource allocation graph into the allocationmatrix and request matrix.

For Resource Preempt the resource

We can snatch one of the resources from the owner of the resource(process) and give it to the other process with the expectation that it will complete the execution and will release this resource sooner.Well, choosing a resource which will be snatched is going to be a bit difficult.

Rollback to a safe state

System passes through various states to get into the deadlock state. The operating system canrollback the system to the previoussafe state. For this purpose, OS needs to implement check pointingat every state.

The moment, we get into deadlock, we will rollback all the allocations to get into the previous safe state.

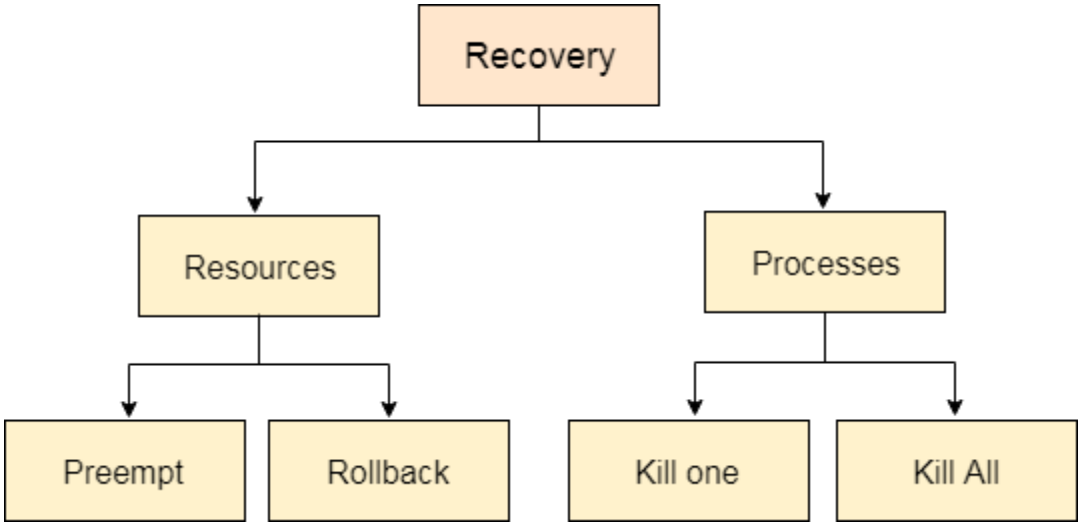
For Process Kill a process

Killing a process can solve our problem but the bigger concern isto decide which process to kill. Generally, Operating system kills a process which has done least amount of work until now.

Kill all process

This is not a suggestible approach but can be implemented if the problem becomes very serious. Killing all process will lead to inefficiency in the system because all the processes will execute again from starting.

Downloaded by bhagyashri 9411 (bhagyashri94112008@gmail.com)



Deadlock issues in deadlock detection & resolution:

Table of Contents

- Deadlock
  - Deadlock Detection:
    - 1. Resource Allocation Graph (RAG) Algorithm:
    - 2. Resource-Requesting Algorithms:
  - Deadlock Resolution:
    - 1. Deadlock Prevention:
    - 2. Deadlock Avoidance:
    - 3. Deadlock Detection with Recovery:

Deadlock

- Deadlock is a fundamental problem in distributed systems.
- A process may request resources in any order, which may not be known a priori and a process can request resource while holding others.
- If the sequence of the allocations of resources to theprocesses is not controlled.
- A deadlock is a state where a set of processes requestresources that are held by other processes in the set.

DEADLOCK DETECTION:

1. Resource Allocation Graph (RAG) Algorithm:

- Deadlock detection typically involves constructing a resource allocation graph based on the current resourceallocation and request status.
- The RAG algorithm identifies cycles in the graph,indicating the presence of a potential deadlock.
- However, the RAG algorithm suffers from scalabilityissues in large systems due to the overhead of maintaining the graph.

2. Resource-Requesting Algorithms:

- Another approach is to periodically check the state of resource requests and allocations to identify potential deadlocks.
- This approach involves tracking the [illegible] tion state and examining

This document is available on studocu
Downloaded by bhagyashri 9411 (bhagyashri9411@2008@gmail.com)

resource requests to detect circularwaits.

- However, this method may have high overhead and canonly identify deadlocks when they occur during the detection phase.

DEADLOCK RESOLUTION:

1. Deadlock Prevention:

- Prevention involves ensuring that at least one of the necessary conditions for deadlock (mutual exclusion,hold and wait, no preemption, circular wait) is not satisfied.
- By carefully managing resource allocation and enforcingcertain policies, deadlocks can be avoided altogether.
- However, prevention methods can be complex, restrictive,and may limit system performance or resource utilization.

2. Deadlock Avoidance:

- Avoidance involves dynamically analyzing resource requests and allocations to ensure that the system avoidsentering an unsafe state where a deadlock can occur.
- Resource allocation is made based on resource requirement forecasts and resource availability to preventcircular waits.
- Avoidance requires a safe state detection algorithm to determine if a resource allocation will lead to a deadlock.
- However, avoidance techniques may suffer from increased overhead and may limit system responsiveness.

3. Deadlock Detection with Recovery:

- Deadlock detection algorithms can be used to periodically check the system's state for potential deadlocks.
- Once a deadlock is detected, recovery mechanisms can beemployed to resolve the deadlock.
- Recovery may involve aborting one or more processes, rolling back their progress, and reallocating resources toallow the system to continue.
- However, recovery mechanisms can be complex and mayresult in data loss or system instability.

Distributed File System:

The distributed file system in the operating system and its features, components, advantages, and disadvantages.

What is Distributed File System?

A **distributed file system (DFS)** is a file system that is distributed on various file servers and locations. It permits programs to accessand store isolated data in the same method as in the local files. It also permits the user to access files from any system. It allows

Downloaded by bhagyashri 9411 (bhagyashri9411@gmail.com)

network users to share information and files in a regulated and permitted manner.

DFS's primary goal is to enable users of physically distributed systems to share resources and information through the **Common File System (CFS)**. It is a file system that runs as a part of the operating systems. Its configuration is a set of workstations and mainframes that a LAN connects. The process of creating a namespace in DFS is transparent to the clients.

DFS has two components in its services, and these are as follows:

1. **Local Transparency**
2. **Redundancy**

Local Transparency

It is achieved via the namespace component.Redundancy

- It is achieved via a file replication component.
- In the case of failure or heavy load, these components work together to increase data availability by allowing data from multiple places to be logically combined under a single folderknown as the **"DFS root"**.
- It is not required to use both DFS components simultaneously; the namespace component can be used without the file replication component, and the file replication component can be used between servers without the namespace component.

Features

There are various features of the DFS. Some of them are as follows:Transparency

There are mainly four types of transparency. These are as follows:

1. **Structure Transparency**

The client does not need to be aware of the number or location of file servers and storage devices. In structure transparency, multiple file servers must be given to adaptability, dependability, and performance.

2. **Naming Transparency**

There should be no hint of the file's location in the file's name. When the file is transferred form one node to other, the file name should not be changed.

3. **Access Transparency**

Local and remote files must be accessible in the same method. The file system must automatically locate the accessed file and deliver it to the client.

4. **Replication Transparency**

When a file is copied across various nodes, the copies files and their locations must be hidden from one node to the next.

Scalability

This document is available on studocu

Downloaded by bhagyashri 9411 (bhagyashri94112008@gmail.com)

The distributed system will inevitably increase over time when more machines are added to the network, or two networks are linked together. A good DFS must be designed to scale rapidly as the system's number of nodes and users increases.

Data Integrity

Many users usually share a file system. The file system needs to secure the integrity of data saved in a transferred file. A concurrency control method must correctly synchronize concurrent access requests from several users who are competing for access to the same file. A file system commonly provides users with atomic transactions that are high-level concurrencymanagement systems for data integrity.

High Reliability

The risk of data loss must be limited as much as feasible in an effective DFS. Users must not feel compelled to make backups of their files due to the system's unreliability. Instead, a file system should back up key files so that they may be restored if the originals are lost. As a high-reliability strategy, many file systems use stable storage.

High Availability

A DFS should be able to function in the case of a partial failure, like a node failure, a storage device crash, and a link failure.

Ease of Use

The UI of a file system in multiprogramming must be simple, and the commands in the file must be minimal.

Performance

The average time it takes to persuade a client is used to assess performance. It must perform similarly to a centralized file system.

Distributed File System Replication

Initial versions of DFS used **Microsoft's File Replication Service (FRS)**, enabling basic file replication among servers. FRS detects new or altered files and distributes the most recent versions of the full file to all servers.

Windows Server 2003 R2 developed the "**DFS Replication" (DFSR)**. It helps to enhance FRS by only copying the parts of files that have changed and reducing network traffic with data compression. It also gives users the ability to control networktraffic on a configurable schedule using flexible configuration options.

History of Distributed File System

The DFS's server component was firstly introduced as an additional feature. When it was incorporated into **Windows NT 4.0Server**, it was called **"DFS 4.1"**. Later, it was declared a standard component of all **Windows 2000 Server** editions. **Windows NT 4.0** and later versions of Windows have client-side support.

Working of Distributed File System

There are two methods of DFS in which they might be implemented, and these are as follows:

Downloaded by bhagyashri9411 (bhagyashri9411@gmail.com)

1. **Standalone DFS namespace**
2. **Domain-based DFS namespace**

Standalone DFS namespace

It does not use Active Directory and only permits DFS roots that exist on the local system. A Standalone DFS may only be acquired on the systems that created it. It offers no-fault liberation and maynot be linked to other DFS.

Domain-based DFS namespace

It stores the DFS configuration in Active Directory and creating namespace root at **domainname>dfsroot>** or **FQDN>dfsroot>**.

DFS namespace

SMB routes of the form are used in traditional file shares that are linked to a single server.

```
\\<SERVER>\<path>\<subpath>
```

Domain-based DFS file share paths are identified by utilizing thedomain name for the server's name throughout the form.

```
\\<DOMAIN.NAME>\<dfsroot>\<path>
```

When users access such a share, either directly or through mapping a disk, their computer connects to one of the accessible servers connected with that share, based on rules defined by the network administrator. For example, the default behavior is for users to access the nearest server to them; however, this can be changed to prefer a certain server.

Applications of Distributed File System

There are several applications of the distributed file system. Some of them are as follows:

Hadoop

Hadoop is a collection of open-source software services. It is a software framework that uses the MapReduce programming style to allow distributed storage and management of large amounts of data. Hadoop is made up of a storage component known as **Hadoop Distributed File System (HDFS)**. It is an operational component based on the MapReduce programming model.

NFS (Network File System)

A client-server architecture enables a computer user to store, update, and view files remotely. It is one of various DFS standardsfor Network-Attached Storage.

SMB (Server Message Block)

IBM developed an SMB protocol to file sharing. It was developed topermit systems to read and write files to a remote host across a LAN. The remote host's directories may be accessed through SMB and are known as **"shares"**.

This document is available on studocu

Downloaded by bhagyashri 9411 (bhagyashri9411@gmail.com)

Design Issues of Distributed System:

Distributed System is a collection of autonomous computer systems that are physically separated but are connected by a centralized computer network that is equipped with distributed system software. These are used in numerous applications, such as online gaming, web applications, and cloud computing. However, creating a distributed system is not simple, and there are a number of design considerations to take into account. The following are some of the major design issues of distributed systems:

Design Issues of Distributed Systems

1. Heterogeneity
2. Openness
3. Security
4. Synchronization
5. Absence of global clock
6. Partial failures
7. Scalability
8. transparency

1. Heterogeneity:

- The distributed system contains many different kinds of hardware and software working together in cooperative fashion to solve problems.
- There may be many different representations of data in the system this might include different representations for integers, byte streams, floating point numbers and character sets.
- There may be many different instructions sets. An application compiled for one instruction set cannot be easily run on a computer with another instruction set unless an instruction set interpreter is provided
- Components in the distributed system have different capabilities like faster clock cycles, larger memory capacity, bigger disk farms, printers and other peripherals and different services

High Degree of node heterogeneity:

- High-performance parallel systems (multiprocessors as well as multicomputer)
- High-end PCs and workstations (servers)
- Simple network computers (offer users only networkaccess)
- Mobile computers (palmtops, laptops)
- Multimedia workstations High degree of network heterogeneity:
- Local area gigabit networks
- Wireless connections
- Long-haul, high-latency connections

2. Openness:

- The openness of a computer system is the characteristic that determines whether the system can be extended and reimplemented in various ways
- The challenge to designers is to tackle the complexity of distributed systems consisting of many components engineered by different people
- Open systems are characterized by the fact that their key interfaces are published\
- Open distributed systems are based on the provision of a uniform communication mechanism and published interfaces for access to shared resources

Downloaded by bhagyashri9411 (bhagyashri94112000@gmail.com)

- Open distributed systems can be constructed from heterogeneous hardware and software, possibly fromdifferent vendors

3. Security

- Shared data must be protected

❖ Privacy - avoid unintentional disclosure of private data

❖ Security – data is not revealed to unauthorized parties

❖ Integrity – protect data and system state from corruption

- Denial of service attacks – put significant load on the system, prevent users from accessing it

Security in detail concerned in the following areas:

❖ Authentication, Authorization/Access control: are the means to identify the right user and user right.

❖ Critical Infrastructure Protection: CIP is the protection of information systems for critical infrastructures including telecommunications, energy, financial services, manufacturing, water, transportation, health care and emergency services sectors

❖ Distributed Trust and Policy Management: designed to address the authorization needs for the next-generation distributed systems. A trust management system is a term coined to refer to a unified framework for the speciation of security policies, the representation of credentials, and the evaluation and enforcement of policy compliances.

❖ Multicasting security and IPR Protection: defines the common architecture for multicast security(MSEC) key management protocols to support a variety of application, transport, network layer security protocol and the intellectual property rights

❖ Multimedia Security: is intended to provide an advanced multimedia application course with its focus on security. two major areas of concern- to ensure secure uses of multimedia data and to use multimedia data for security applications

❖ Risk analysis, Assessment, Management: A security policy framework is necessary to support the security infrastructure required for the secure movement of sensitive information across and within national boundaries

Synchronization:

1. Concurrent cooperating tasks need to synchronize
   - When accessing shared data
   - When performing a common task
2. Synchronization must be done correctly to prevent data corruption:
   - Example: two account owner; one deposits the money,the other one withdraws; they act concurrently
   - How to ensure the bank account is in "correct" stateafter these actions?
3. Synchronization implies communication
4. Communication can take a long time
5. Excessive synchronization can limit effectiveness and scalability of distribute system

Absence of Global Clock:

- Cooperating task need to agree on the order of events

This document is available on studocu
Downloaded by bhagyashri 9111 (bhagyashri91112002@gmail.com)

- Each task its own notion of time
- Clocks cannot be perfectly synchronized
- How to determine which even occurred first?

Example:

Bank account, starting balance = $100
Client at bank machine A makes a deposit of $150 Client at bank machine B makes a withdrawal of $100Which event happened first?
Should the bank charge the overdraft fee?

Partial Failures:

- Detection of failures - may be impossible
- Has a component crashed? Or is it just show?
- Is the network down? Or is it just slow?
- If it's slow – how long should we wait?
- Handling of failures
- Re-transmission
- Tolerance for failures
- Roll back partially completed task
- Redundancy against failures
- Duplicate network routes
- Replicated databases

Scalability

- Does the system remain effective as of grows?
- As you add more components:
- More synchronization
- More communication à the system runs slowly.
- Avoiding performance bottlenecks:
- Everyone is waiting for a single shared resource

- In a centrally coordinated system, everyone waits for the co-coordinator

Transparency:

Distributed systems designers must hide the complexity of the systems as much as they can. Adding abstraction layer isparticularly useful in distributed systems.
Example: While users hit search in google.com, they never notice that their query goes through a complex process before Google shows them a result

- Concealing the heterogeneous and distributed nature of the system so that it appears to the user like one system

Transparency categories:

Access: access local and remote resources using identical operations (NFS or Samba-mounted file systems)
Location: access without knowledge of location of a resource
(URL's, e-mail)
Concurrency: allow several processes to operate concurrently using shared resources in a consistent fashion (two users simultaneously accessing the bank account)

Downloaded by bhagyashri 9411 (bhagyashri9411@gmail.com)

Transparency categories

Mobility: allow resources to move around
Performance: adaption of the system to varying load situationswithout the user noticing it
Scaling: allow system and applications to expand without need tochange structure of applications or algorithms

Case Study of a Distributed Operating System

Introduction to Amoeba

- Originated at a university in Holland, 1981
- Currently used in various EU countries

- Built from the ground up. UNIX emulation added later

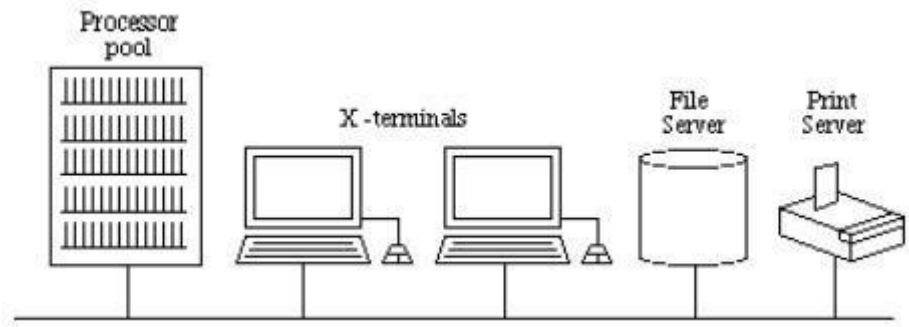
- Goal was to build a transparent distributed operating system

- Resources, regardless of their location, are managed by the system, and the user is unaware of where processes are actually run

The Amoeba System Architectur:

- Assumes that a large number of CPUsare available and that each CPU ha 10s of Mb of memory

- CPUs are organised into processor pools



- CPUs do not need to be of the same architecture (can mix SPARC, Motorola PowerPC, 680x0, Intel, Pentium, etc.)

- When a user types a command, system determines which CPU(s) to execute it on. CPUs can be timeshared.

This document is available on studocu
Downloaded by bhagyashri 9411 (bhagyashri9411@gmail.com)

- Terminals are X-terminals or PCs running X emulators

- The processor pool doesn't have to be composed of CPU boards enclosed in a cabinet, they can be on PCs, etc., in different rooms, countries,...

- Some servers (e.g., file servers) run on dedicated processors, because they need to be available all the time.

<u>The Amoeba Microkernel:</u>

- The Amoeba microkernel is used on all terminals (with an on-board processor), processors, and servers

- The microkernel

  - manages processes and threads
  - provides low-level memory management support
  - supports interprocess communication (point-to-pointand group)
  - handles low-level I/O for the devices attached to themachine

<u>The Amoeba Servers: Introduction:</u>

- OS functionality not provided by the microkernel is performed by Amoeba servers

- To use a server, the client calls a stub procedure which marshalls parameters, sends the message, and blocks until the result comes back

<u>Server Basics:</u>

- Amoeba uses capabilities
- Every OS data structure is an object, managed by a server
- To perform an operation on an object, a client performs an RPC with the appropriate server, specifying the object, the operationto be performed and any parameters needed.

Downloaded by bhagyashri 9411 (bhagyashri9411@gmail.com)

- The operation is transparent (client does not know where serveris, nor how the operation is performed)
- Capabilites

- To create an object the client performs an RPC with the server

- Server creates the object and returns a capability

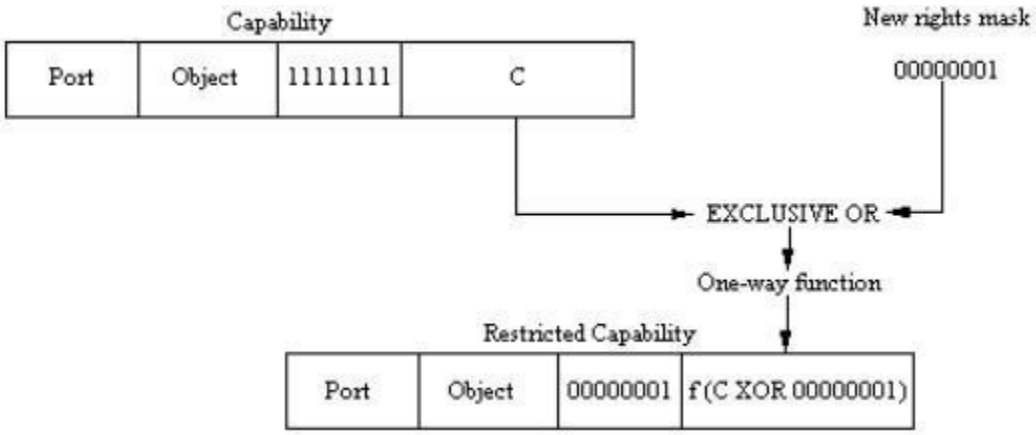
- To use the object in the future, the client must present thecorrect capability

- The check field is used to protect the capability against forgeryObject protection:

- When an object is created, server generates random check field, which it stores both in the capability and in its own tables

- The rights bits in the capability are set to on

- The server sends the owner capability back to the client Creating a capability with restricted rights



- Client can send this new capability to another process

Process Management:

- All processes are objects protected by capabilities
- Processes are managed at 3 levels by process servers, part of the microkernel by library procedures which act as interfaces by the run server, which decides where to run the processes.

- Process management uses process descriptors

This document is available on studocu

Downloaded by bhagyashri 9411 (bhagyashri94112008@gmail.com)

Contains:

- platform description process' owner's capability etc

Memory Management:

- Designed with performance, simplicity and economics inmind
- Process occupies contiguous segments in memory
- All of a process is constantly in memory
- Process is never swapped out or paged

Communication:

- Point-to-point (RPC) and Group

The Amoeba Servers:

The File System

- Consists of the Bullet (File) Server, the Directory Server, andthe Replication Server

The Bullet Server

- Designed to run on machines with large amounts of RAM andhuge local disks
- Used for file storage
- Client process creates a file using the *create* call
- Bullet server returns a capability that can be used to *read* thefile with

- Files are immutable, and file size is known at file creationtime. Contiguous allocation policies used

The Directory Server:

- Used for file naming
- Maps from ASCII names to capabilities
- Directories also protected by capabilities
- Directory server can be used to name ANY object, not justfiles and directories

The Replication Server:

Downloaded by bhagyashri 9411 (bhagyashri9411@gmail.com)

- Used for fault tolerence and performance
- Replication server creates copies of files, when it has time

- Other Amoeba Servers

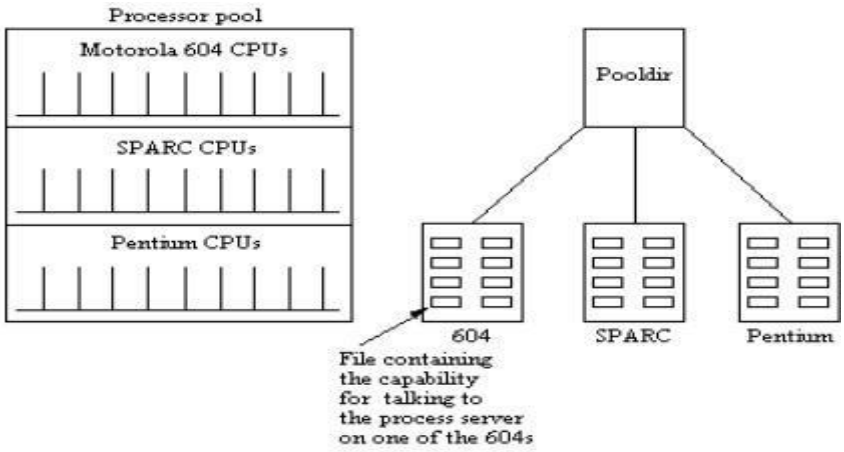
- The Run Server

- When user types a command, two decisions have to be madeo On

which architecture should the process be run?

o Which processor should be chosen?

- Run server manages the processor pools



- Uses processes process descriptor to identify appropriate target architecture

- Checks which of the available processors have sufficient memory to run the process Estimates which of the remaining processor has the most available compute power

<u>The Boot Server:</u>

- Provides a degree of fault tolerance
- Ensures that servers are up and running

- If it discovers that a server has crashed, it attempts to restartit, otherwise selects another processor to provide the service

This document is available on studocu
Downloaded by (kanagavathi0411-(kanagavathi0411@gmail.com))

- Boot server can be replicated to guard against its own failure[i]

SUN Network File System (NFS) :

The Sun Network File System (NFS) has become a common standardfor distributed UNIX file access

- NFS runs over LANs (even over WANs – slowly)

- Basic idea

1. allow a remote directory to be "mounted" (spliced) onto a localdirectory
2. Gives access to that remote directory and all its descendants asif they were part of the local hierarch

- Pretty much exactly like a "local mount" or "link" on UNIX

1. except for implementation and performance …
2. no, we didn't really learn about these, but they're obvious

- For instance:

- I mount /u4/lazowska on Node1 onto /students/foo on Node2
- users on Node2 can then access this directory as /students/foo
- if I had a file /u4/lazowska/myfile, users on Node2 see it as

/students/foo/myfile • Just as, on a local system, I might link /cse/www/education/courses/451/06wi/ as /u4/lazowska/451 to allow easy access to my web data frommy home directory

NFS implementation • NFS defines a set of RPC operations for remotefile access:

- searching a directory
- reading directory entries
- manipulating links and directories
- reading/writing files

- Every node may be both a client and server

- NFS defines new layers in the Unix file system System Call InterfaceVirtual File System buffer cache / i-node table (local files) (remote files) UFS NFS The virtual file system (VFS) provides a standard interface, using v-nodes as file handles. A v-node describes either a local or remote file. RPCs to other (server) nodes RPC requests from remote clients, and server responses

NFS caching / sharing :

- On an open, the client asks the server whether its cached blocksare up to date.
- Once a file is open, different clients can write it and get inconsistentdata.

Downloaded by bhagyashri 9411 (bhagyashri94112008@gmail.com)

- Modified data is flushed back to the server every 30 seconds.

Example: CMU's Andrew File System (AFS)

- Developed at CMU to support all of its student computing
- Consists of workstation clients and dedicated file server machines (differs from NFS)
- Workstations have local disks, used to cache files being used locally (originally whole files, subsequently 64K file chunks) (differs fromNFS)
- Andrew has a single name space – your files have the same names everywhere in the world (differs from NFS)
- Andrew is good for distant operation because of its local disk caching: after a slow startup, most accesses are to local disk.

<u>AFS caching/sharing :</u>

- Need for scaling required reduction of client-server message traffic
- Once a file is cached, all operations are performed locally
- On close, if the file has been modified, it is replaced on the server
- The client assumes that its cache is up to date, unless it receives acallback message from the server saying otherwise

– on file open, if the client has received a callback on the file, it must fetch a new copy; otherwise it uses its locally-cached copy (differs from NFS)

<u>Example: Berkeley Sprite File System:</u>

- Unix file system developed for diskless workstations with largememories at UCB (differs from NFS, AFS)
- Considers memory as a huge cache of disk blocks – memory isshared between file system and VM
- Files are permanently stored on servers – servers have a largememory that acts as a cache as well
- Several workstations can cache blocks for read-only files
- If a file is being written by more than 1 machine, client caching isturned off – all requests go to the server (differs from NFS, AFS)

This document is available on studocu

Downloaded by bhagyashri 9411 (bhagyashri94112008@gmail.com)

Summary :

- There are a number of issues to deal with:

– what is the basic abstraction

- naming
- caching
- sharing and coherency
- replication
- performance

- No right answer! Different systems make different tradeoffs!
- Performance is always an issue

- always a tradeoff between performance and the semantics of fileoperations (e.g., for shared files).

- Caching of file blocks is crucial in any file system

- maintaining coherency is a crucial design issue.

- Newer systems are dealing with issues such as disconnectedoperation for mobile computers.

CODA:

Introduction:

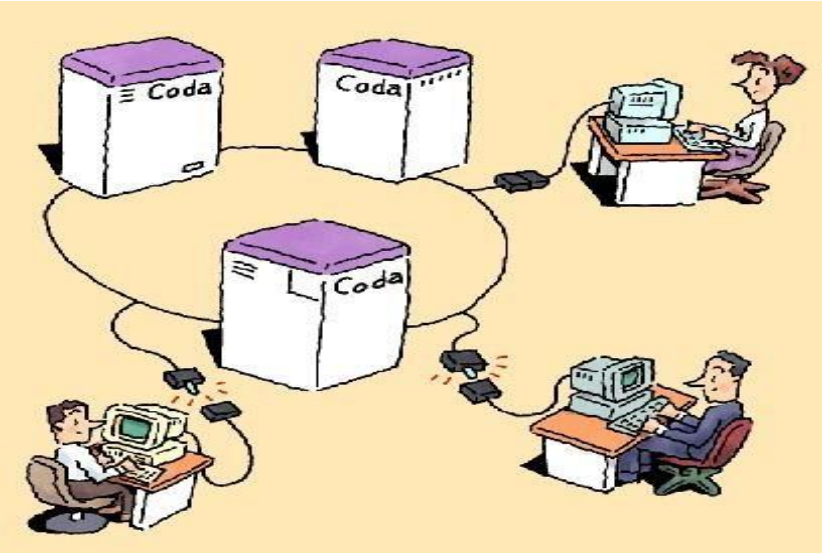
The Coda distributed file system is a state of the art experimentalfile system developed in the group of M. Satyanarayanan at Carnegie Mellon University. Numerous people contributed to Coda which now incorporates many features not found in other systems:Mobile Computing

- disconnected operation for mobile clients
  - reintegration of data from disconnected clients
  - bandwidth adaptation
- Failure Resilience
  - read/write replication servers
  - resolution of server/server conflicts
  - handles of network failures which partition the servers
  - handles disconnection of clients client
- Performance and scalability

  - client side persistent caching of files, directories andattributes for high performance
  - write back caching
- Security
  - kerberos like authentication
  - access control lists (ACL's)

Downloaded by bhagyashri 9411 (bhagyashri94112008@gmail.com)

- Well defined semantics of sharing
- Freely available source code



Distributed file systems:

- A distributed file system stores files on one or more computers called servers, and makes them accessible to other computers called clients, where they appear as normal files.

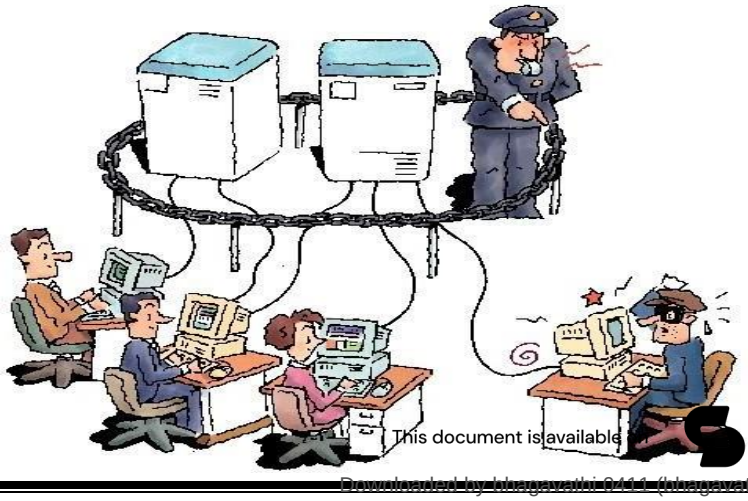
- There are several advantages to using file servers: the files are more widely available since many computers can access the servers, and sharing the files from a single location is easier than distributing copies of files to individual clients.

- Backups and safety of the information are easier to arrange since only the servers need to be backed up.

- The servers can provide large storage space, which might be costly or impractical to supply to every client.

- There are many problems facing the design of a good distributed file system. Transporting many files over the net can easily create sluggish performance and latency, network bottlenecks and server overload can result.

- A few new features are being implemented (write back caching and cells for example) and in several areas components of Coda are being reorganized. We have already received very generous help from users on the net and we hope that this will continue

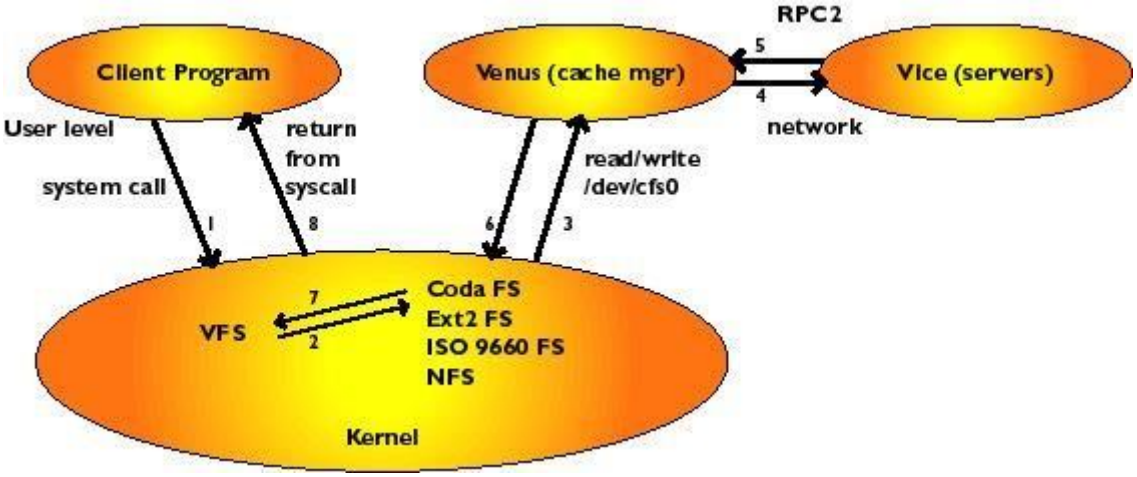


This document is available
studocu
Downloaded by bhagyashri9411 (bhagyashri94112000@gmail.com)

Coda on a client:

- If Coda is running on a client, which we shall take to be a Linux workstation, typing mount will show that a file system --of type ``Coda'' -- mounted under /coda. All the files, which any of the servers may provide to the client, areavailable under this directory, and all clients see the same name space.
- A client connects to ``Coda'' and not to individual servers,which come into play invisibly. This is quite different frommounting NFS file systems which is done on a per server, per export basis.
- In the most common Windows systems (Novell andMicrosoft's CIFS) as well as with Appleshare on theMacintosh, files are also mounted per volume.

- To understand how Coda can operate when the network connections to the server have been severed, let's analyze asimple file system operation. Suppose we type: ``cat /coda/tmp/foo'' to display the contents of a Coda file.



- When the kernel passes the open request to Venus for thefirst time, Venus fetches the entire file from the servers, using remote procedure calls to reach the servers. It then stores the file as a container file in the cache area (currently/usr/coda/venus.cache/).
- The file is now an ordinary file on the local disk, and read-write operations to the file do not reach Venus but are (almost) entirely handled by the local file system (ext2 for Linux).
- Coda read-write operations take place at the same speed as those to local files. If the file is opened a second time, it will not be fetched from the servers again, but the local copy willbe available for use immediately.

From caching to disconnected operation:

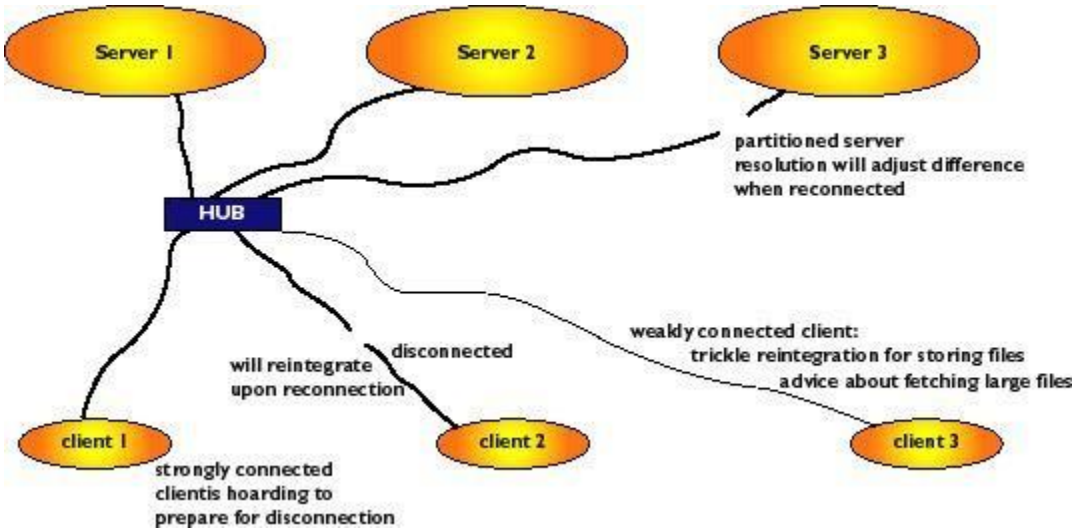
- The origin of disconnected operation in Coda lies in one ofthe original research aims of the project: to provide a file system with resilience to network failures.
- AFS, which supported 1000's of clients in the late 80's on the CMU campus had become so large that network outagesand server failures happening somewhere

Downloaded by bhagyashri9411 (bhagyashri9411@gmail.com)

almost every day became a nuisance.

- It turned out to be a well timed effort since with the rapidadvent of mobile clients (viz. Laptops) and Coda's supportfor failing networks and servers Coda equally applied to mobile clients.

- when the update is complete on the client it has also been made on the server. If a server is unavailable, or if the network connections between client and server fail, such anoperation will incur a time-out error and fail.
- To support disconnected computers or to operate in the presence of network failures, Venus will not report failure(s)to the user when an update incurs a time-out.
- To support disconnected computers or to operate in the presence of network failures, Venus will not report failure(s)to the user when an update incurs a time-out.
- The second issue is that during reintegration it may appear that during the disconnection another client has modified the file too and have shipped it to the server. This is called alocal/global



Volumes, Servers and Server Replication:

- Files on Coda servers are not stored in traditional file systems. These partitions will contain files which are grouped into volumes. Each volumehas a directory structure like a file system: i.e. a root directory for the volume and a tree below it.

- Typically a single server would have some hundreds of volumes, perhapswith an average size approximately 10MB. A volume is a manageable amount of file data which is a very natural unit from the perspective of system administration and has proven to be quite flexible.

- Coda holds volume and directory information, access control lists and fileattribute information in raw partitions. These are accessed through a log based recoverable virtual memory package (RVM) for speed andconsistency.

- Only file data resides in the files in server partitions. RVM has built in support for transactions - this means that in case of a server crash thesystem can be restored to a consistent state without much effort.

- Coda identifies a file by a triple of 32bit integers called a Fid: it consists of a VolumeId, a VnodeId and a Uniquifier. The VolumeId identifies the volume in which the file resides. The VnodeId is the ``inode'' number of the file, andthe uniquifiers are needed for resolution. The Fid is

This document is available on studocu
Downloaded by bhagyashri9411 (bhagyashri9411@gmail.com)

unique in a cluster of Coda servers.

- The advantage of this is higher availability of data: if one server fails otherstake over without a client noticing the failure. Volumes can be stored on agroup of servers called the VSG (Volume Storage Group).

Coda in action:

Coda is in constant active use at CMU. Several dozen clients use it for development work (of Coda), as a general-purpose file system and for specificdisconnected applications. The following two scenarios have exploited the features of Coda very successfully.

There are a number of compelling future applications where Coda could providesignificant benefits.

- WWW replication servers should be Coda clients. Many ISPs are struggling with a few WWW replication servers. They have too much access to use justa single http server. Using NFS to share the documents to be served has proven problematic due to performance problems, so manual copying of files to the individual servers is frequently done.

- Network computers could exploit Coda as a cache to dramatically improveperformance. Updates to the network computer would automatically be made when as they become available on servers, and for the most part thecomputer would operate without network traffic, even after restarts.

Getting Coda:

Coda is available for ftp from ftp.coda.cs.cmu.edu. You will find RPM packages forLinux as well as tar balls with source. Kernel support for Coda will come with the Linux 2.2 kernels. On the WWW site www.coda.cs.cmu.edu you will find additional resources such as mailing lists, manuals and research papers.

Downloaded by bhagyashri 9411 (bhagyashri9411@gmail.com)

SHANMUGA INDUSTRIES ARTS & SCIENCE COLLEGE,T.V.MALAI.

PG &RESEARCH DEPARTMEMT OF COMPUTER SCIENCE

CLASS : I MSc Computer Science

SUBJECT CODE :

SUBJECT NAME : Advanced Operating System

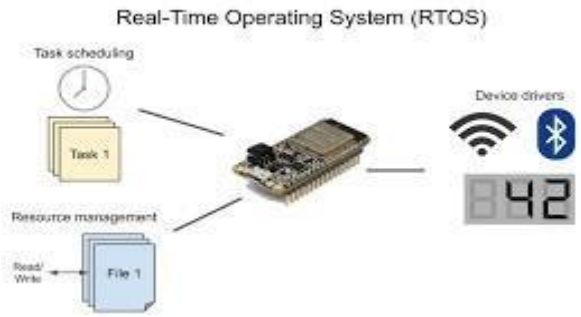
UNIT III

REALTIME OPERATING SYSTEM

Real time operating system: Introduction – Applications of real time system – Basic Model of real time system – characteristics – safety and reliability – real time task scheduling.

Real time operating system:

Real-time **operating systems (RTOS)** are used in environments where a large number of events, mostly external to the computer system, must be accepted andprocessed in a short time or within certain deadlines.



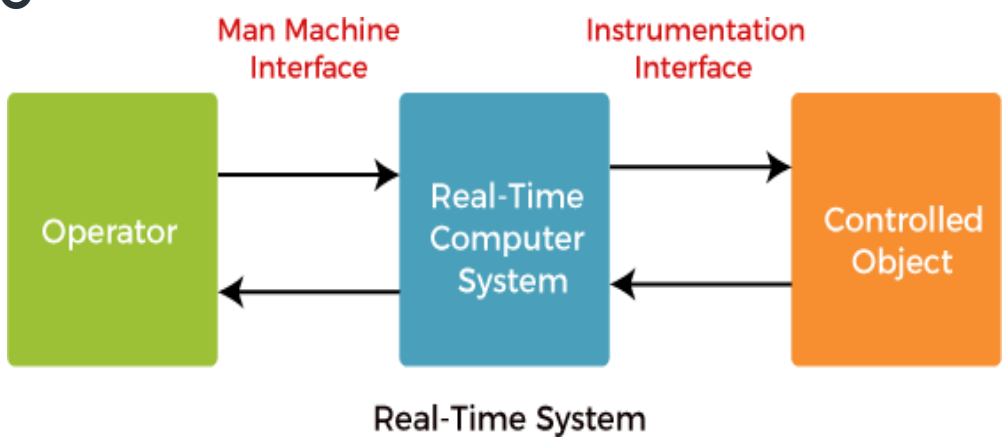
Such applications are industrial control, telephone switching equipment, flight control, and real-time simulations. With an RTOS, the processing time is measured in tenths of seconds.

This system is time-bound and has a fixed deadline. The processing in this type ofsystem must occur within the specified constraints. Otherwise, this will lead to system failure.

Example: of real-time operating systems are airline traffic control systems, Command Control Systems, airline reservation systems, Heart pacemakers,Network Multimedia Systems, robots, etc.

This document is available on studocu

Downloaded by bhargavathi 9411 (bhargavathi9411 2000@gmail.com)



Real-Time System is used at those Places where we require higher and timely responses. Real-time operating systems involve a set of applications where the operations are performed on time to run the activities in an external system. It uses the quantitative expression of time to analyze the system's performance.

The ***deadline*** in the context of a real-time system is the moment of the time by whichthe job's execution is needed to be accomplished.

Most real-time operating systems use a ***pre-emptive scheduling*** algorithm.

Advantages of Real-Time Operating Systems:

The real-time operating system has the following advantages, such as:

- **Task switching:**

 Real-time operating systems are designed, so that starting switching is a very quick process. A normal traditional operating system takes a lot of time while switchingfrom one person to another process.

Downloaded by bhargavatti 9411 (bhargavatti9411@gmail.com)

- The real-time operating system completes the task switching only within some ofthe microseconds.

- **Focus execution:**

The main focus of real-time operating systems is to handle the task inexecution, and just a little focus is on the task in waiting.

- **Error-free:**

- Real-time operating systems are designed very carefully, and different kinds of software testing techniques are applied to the real-time operating system to test thesystem.
- This detailed testing makes the real-time operating system approximately errorfree.

- **Maximum resource utilization:**

- Real-time operating systems are designed to follow the task within a given timesecretly. This is not simple and easy to complete the task within time.
- The real-time operating system utilizes all the hardware efficiently and completely. The main focus of the real-time operating system is not to save energy or resources.
- So we can see that the real-time operating system utilize the resources completelyand very efficiently.

- **Usage in embedded systems:**

Real-time operating systems can also work with embedded systems.

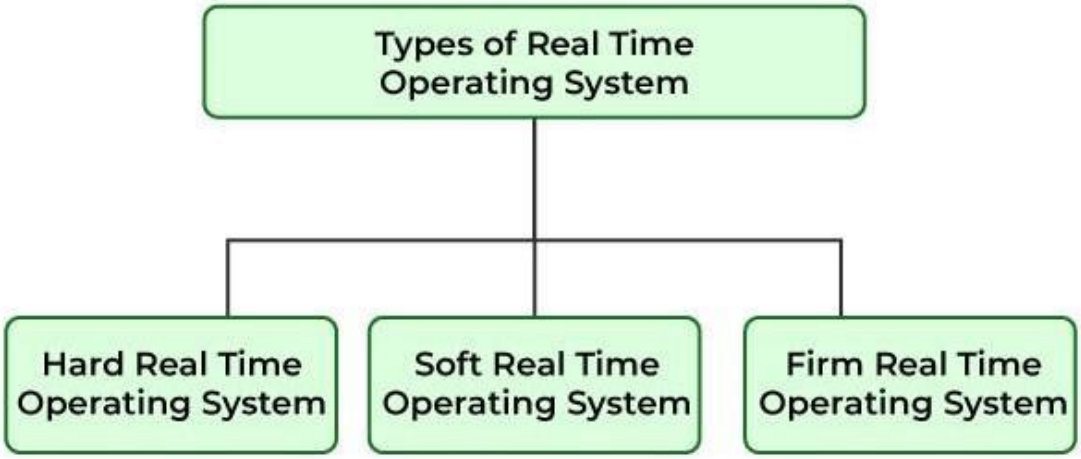
- **All-time performance:**

Real-time operating systems are designed to works 24 hours a day and
every day.

<u>The real-time operating systems can be of 3 types:</u>

This document is available on studocu

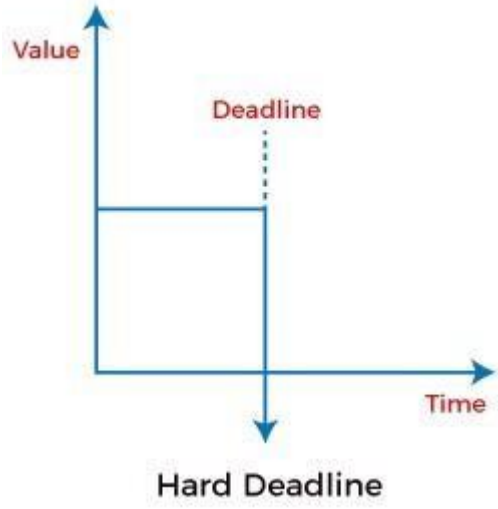
Downloaded by bhagavathi 9411 (bhagavathi9411 2000@gmail.com)



Hard Real-Time Operating System:

These operating systems guarantee that critical tasks are completed within a rangeof time.

Example: a robot is hired to weld a car body. If the robot welds too early or too late,the car cannot be sold, so it is a hard real-time system that requires complete car welding by the robot hardly on time., scientific experiments, medical imaging systems, industrial control systems, weapon systems, robots, air traffic control systems, etc.



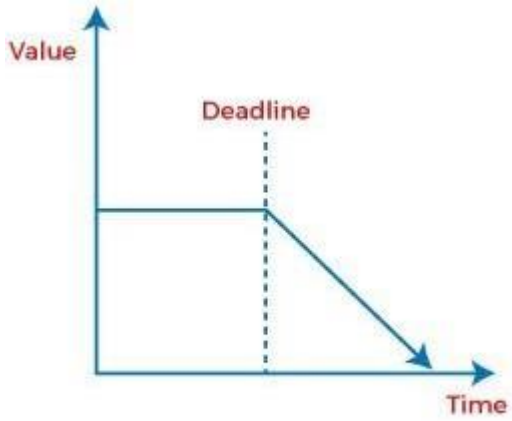
Downloaded by bhargavi 9411 (bhargavi9411 2000@gmail.com)

Soft real-time operating system:

This operating system provides some relaxation in the time limit. Example: Multimedia systems, digital audio systems, etc. Explicit,

programmerdefined, and controlled processes are encountered in real-time systems.

A separate process is changed by handling a single external event. The process is activated upon the occurrence of the related event signaled by an interrupt.



Soft Deadline

Firm Real-time Operating System:

RTOS of this type have to follow deadlines as well. In spite of its small impact, missing a deadline can have unintended consequences, including a reduction in the quality of the product. Example: Multimedia applications.

Deterministic Real-time operating System:

Consistency is the main key in this type of real-time operating system. It ensures that all the task and processes execute with predictable timing all the time,which make it more suitable for applications in which timing accuracy is very important.**Examples:** INTEGRITY, PikeOS.

Applications of Real-time System:Industrial application:

Real-time system has a vast and prominent role in modern industries. Systems are made real time based so that maximum and accurate output can be obtained.

- In order to such things real -time systems are used in maximum industrial organizations. These system somehow lead to better performance and highproductivity in less time.

Example: Automated Car Assembly Plant, Chemical Plants,

This document is available on studocu

Downloaded by bhargavathi 9411 (bhargavathi9411 2003@gmail.com)

Medical Science application:

- In the field of medical science, real-time system has a huge impact on the humanhealth and treatment.
- Due to the introduction of real-time system in medical science, many lives aresaved and treatment of complex diseases has been turned down to easier ways.

- People specially related to medical, now feel more relaxed due to these systemExample: Robot, MRI Scan, Radiation therapy etc.

Peripheral Equipment applications:

- Real-time system has made the printing of large banners and such things veryeasier. Once these systems came into use, the technology world became morestrong.

- Peripheral equipment are used for various purposes. These systems are embedded with micro chips and perform accurately in order to get the desiredresponse.

Example: Laser printer, fax machine, digital camera etc.

Telecommunication applications:

Real-time system map the world in such a way that it can be connected within a short time. Real-time systems have enabled the whole world to connect via a medium across the internet.

- These systems make the people connect with each other in no time and feel the real environment of togetherness.

Example: Video Conferencing, Cellular system etc.

Defense applications:

- In the new era of atomic world, defense is able to produce the missiles whichhave the dangerous powers and have the great destroying ability.

- All these systems are real-time systems, and it provides the system to attack anda system to defend.

Example: Missile guidance system, anti-missile system, Satellite missile systemetc.

Aerospace applications:

- The most powerful use of real time system is in aerospace applications.Basically hard real time systems

Downloaded by bhagavathi 9411 (bhagavathi9411@gmail.com)

are used in aerospace applications.

- here the delay of even some second is not allowed and if it happens, systemfails.

Example: Satellite tracking system, Avionics, Flight simulation etc.

Basic Model of a Real-time System:

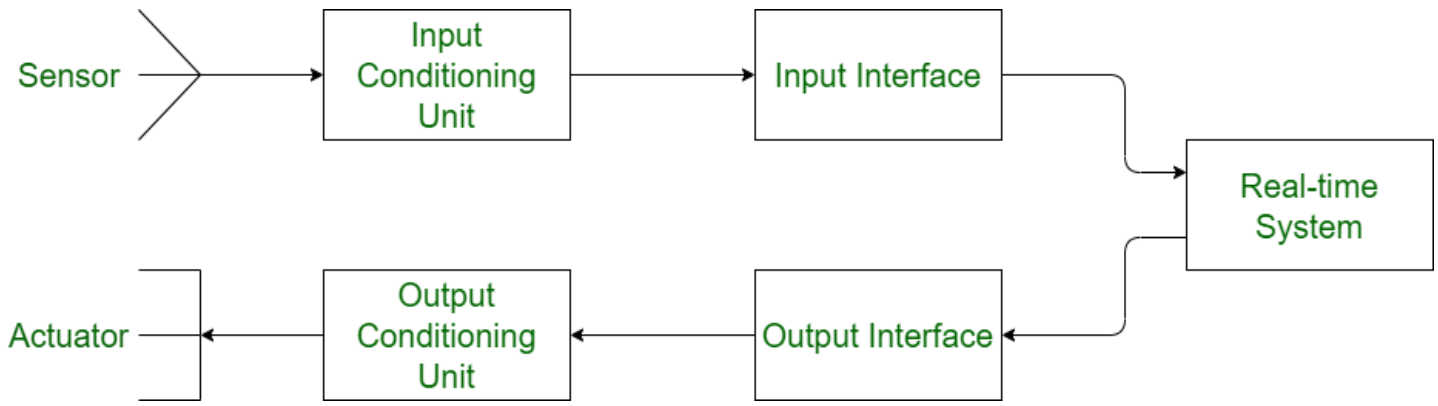
- The basic model of a real-time system presents the overview of all thecomponents involved in a real-time system.

- Real-time system includes various hardware and software embedded in a such away that the specific tasks can be performed in the time constraints allowed.

- The accuracy and correctness involved in real-time system makes the model complex. There are various models of real-time system which are more complexand are hard to understand.

Here we will discuss a basic model of real-time system which has some commonly used terms and hardware.

Following diagram represents a basic model of Real-time System:



Sensor:

- Sensor is used for the conversion of some physical events or characteristics intothe electrical signals. These are hardware devices that takes the input from environment and gives to the system by converting it.

Example: a thermometer takes the temperature as physical characteristic and thenconverts it into electrical signals for the system.

Actuator:

- Actuator is the reverse device of sensor. Where sensor converts the physical events into electrical signals, actuator does the reverse. It converts the electrical signals into the physical events or characteristics.

This document is available on studocu
Downloaded by bhagavathi 8411 (bhagavathi8411 2000@gmail.com)

- It takes the input from the output interface of the system. The output from theactuator may be in any form of physical action.

- Some of the commonly used actuator are motors and heaters.

Signal Conditioning Unit:

- When the sensor converts the physical actions into electrical signals, then computer can't used them directly. Hence, after the conversion of physicalactions into electrical signals, there is need of conditioning. Similarly while giving the output when electrical signals are sent to the actuator, then also conditioning is required.

Therefore, Signal conditioning is of two types:

- **Input Conditioning Unit:** It is used for conditioning the electrical signalscoming from sensor.
- **Output Conditioning Unit:** It is used for conditioning the electrical signalscoming from the system.

Interface Unit:

Interface units are basically used for the conversion of digital to analog and viceversa. Signals coming from the input conditioning unit are analog and the system does the operations on digital signals only, then the interface unit is used tochange the analog signals to digital signals.

On this basis, Interface unit is also of two types:

- **Input Interface:** It is used for conversion of analog signals to digital.
- **Output Interface:** It is used for conversion of digital signals to analog.

Characteristics of Real-time System:

Following are the some of the characteristics of Real-time System:

1. Time Constraints:

- Time constraints related with real-time systems simply means that time intervalallotted for the response of the ongoing program.

- This deadline means that the task should be completed within this time interval. Real-time system is responsible for the completion of all tasks within their timeintervals.

Downloaded by bhagavathi 8411 (bhagavathi8411 2000@gmail.com)

2. Correctness:

- Correctness is one of the prominent part of real-time systems. Real-time systemsproduce correct result within the given time interval.

If the result is not obtained within the given time interval then also result is not considered correct. In real-time systems, correctness of result is to obtain correctresult in time constraint.

3. Embedded:

- All the real-time systems are embedded now-a-days. Embedded system means that combination of hardware and software designed for a specific purpose.

- Real-time systems collect the data from the environment and passes to othercomponents of the system for processing.

4. Safety:

- Safety is necessary for any system but real-time systems provide critical safety.Real-time systems also can perform for a long time without failures.

- It also recovers very soon when failure occurs in the system and it does notcause any harm to the data and information.

5. Concurrency:

- Real-time systems are concurrent that means it can respond to a several numberof processes at a time.
- There are several different tasks going on within the system and it respondsaccordingly to every task in short intervals.
- This makes the real-time systems concurrent systems.

6. Distributed:

- In various real-time systems, all the components of the systems are connected ina distributed way.
- The real-time systems are connected in such a way that different components are at different geographical locations.
- Thus all the operations of real-time systems are operated in distributed ways.

This document is available on studocu

Downloaded by bhargavatti 9411 (bhargavatti9411@gmail.com)

7. Stability:

Even when the load is very heavy, real-time systems respond in the time constraint i.e. real-time systems does not delay the result of tasks even whenthere are several task going on a same time.

- This brings the stability in real-time systems.

8. Fault tolerance:

- Real-time systems must be designed to tolerate and recover from faults or errors.The system should be able to detect errors and recover from them without affecting the system's performance or output.

9. Determinism:

- Real-time systems must exhibit deterministic behavior, which means that thesystem's behavior must be predictable and repeatable for a given input.
- The system must always produce the same output for a given input, regardless of the load or other factors.

10. Real-time communication:

- Real-time systems often require real-time communication between differentcomponents or devices.
- The system must ensure that communication is reliable, fast, and secure.

11. Resource management:

- Real-time systems must manage their resources efficiently, including processing power, memory, and input/output devices.
- The system must ensure that resources are used optimally to meet the time constraints and produce correct results.

12. Heterogeneous environment:

- Real-time systems may operate in a heterogeneous environment, where differentcomponents or devices have different characteristics or capabilities.

Downloaded by bhargavatti 8411 (bhargavatti8411 2003@gmail.com)

 The system must be designed to handle these differences and ensure that allcomponents work together seamlessly.

13. Scalability:

Real-time systems must be scalable, which means that the system must be able tohandle varying workloads and increase or decrease its resources as needed.

14. Security:

 Real-time systems may handle sensitive data or operate in criticalenvironments, which makes security a crucial aspect.

 The system must ensure that data is protected and access is restricted toauthorized users only.

Safety and reliability:

 Safety and reliability are critical aspects of real-time systems, especially thoseused in mission-critical applications such as aerospace, automotive, medical devices, and industrial control systems.

 Real-time systems are those that must meet strict timing constraints, where the correctness of the system not only depends on the logical result of computationsbut also on the timing of these results.

Here are some key considerations for ensuring safety and reliability in real-timesystems:

 Redundancy:

Implementing redundancy in critical components of the system can enhance reliability. This could involve duplicating hardware components, introducing votingmechanisms, or using diverse redundancy to mitigate common-mode failures.

 Fault Tolerance:

Real-time systems should be designed to tolerate faults gracefully. This includes mechanisms for detecting faults, isolating faulty components, and reconfiguring thesystem to maintain critical functionality.

 Deterministic Behavior:

Ensuring deterministic behavior is crucial in real-time systems. Predictable execution paths, bounded latencies, and minimal jitter are essential for meetingtiming constraints.

This document is available on studocu
Downloaded by bhagavathi 8411 (bhagavathi8411 [illegible]@gmail.com)

- **Validation and Verification:**

Rigorous validation and verification processes are necessary to ensure that the system meets safety and reliability requirements. This involves thorough testing, simulation, and formal verification techniques to detect and eliminate potential hazards.

- **Safety Standards Compliance:**

Adhering to industry-specific safety standards and regulations is essential for ensuring the safety and reliability of real-time systems. Standards such as ISO26262 for automotive, DO-178C for aerospace, and IEC 61508 for industrial control systems provide guidelines for developing safe and reliable systems.

- **Real-time Operating Systems (RTOS):**

Using an RTOS tailored for real-time applications can provide features such as priority-based scheduling, deterministic task execution, and resource management, which are essential for building reliable real-time systems.

- **Hardware Considerations:**

Selecting appropriate hardware components with built-in safety features, such as error-correcting memory, redundant processors, and hardware watchdogs, can contribute to the overall reliability of the system.

- **Failure Analysis and Mitigation:**

Incorporating mechanisms for failure analysis and mitigation can help identify potential hazards and minimize their impact on system operation. This includes techniques such as failure mode and effects analysis (FMEA) and fault tree analysis(FTA).

- **Update and Maintenance Procedures:**

Establishing procedures for software updates, patches, and maintenance activities is crucial for maintaining the safety and reliability of real-time systems over their operational lifespan.

- **Continuous Monitoring and Diagnostics:**

Implementing continuous monitoring and diagnostics can help detect and respond to abnormal system behavior in real-time, preventing potential safety hazards and ensuring reliable operation.

By integrating these principles and practices into the design, development, and deployment of real-time systems, engineers can enhance their safety and reliability, meeting the stringent requirements of mission-critical applications.

Real time task scheduling:

Downloaded by bhagavathi 8411 (bhagavathi8411@gmail.com)

Real-time systems are systems that carry real-time tasks. These tasks need to be performed immediately with a certain degree of urgency. In particular, these tasks are related to control of certain events (or) reacting to them.

Real-time tasks can be classified as :

- **hard real-time tasks**
- **soft real-time tasks.**

A hard real-time task must be performed at a specified time which could otherwise lead to huge losses. In soft real-time tasks, a specified deadline can be missed. Thisis because the task can be rescheduled (or) can be completed after the specified time,

In real-time systems, the scheduler is considered as the most important component which is typically a short-term task scheduler. The main focus of this scheduler is to reduce the response time associated with each of the associated processes instead of handling the deadline.

If a preemptive scheduler is used, the real-time task needs to wait until its corresponding tasks time slice completes. In the case of a non-preemptive scheduler, even if the highest priority is allocated to the task, it needs to wait untilthe completion of the current task. This task can be slow (or) of the lower priorityand can lead to a longer wait.

A better approach is designed by combining both:

- Preemptive
- non-preemptive scheduling.

This can be done by introducing time-based interrupts in priority based systems which means the currently running process is interrupted on a time-based intervaland if a higher priority process is present in a ready queue, it is executed by preempting the current process.

Based on schedulability, implementation (static or dynamic), and the result (self ordependent) of analysis,

The scheduling algorithm are classified as follows:

1. **Static table-driven approaches:**

These algorithms usually perform a static analysis associated with schedulingand capture the schedules that are advantageous. This helps in providing a schedule that can point out a task with which the execution must be started atrun time.

2. **Static priority-driven preemptive approaches:**

This document is available on studocu

Downloaded by bhagavathi 9411 (bhagavathi9411 2000@gmail.com)

Similar to the first approach, these type of algorithms also uses static analysis ofscheduling. The difference is that instead of selecting a particular schedule, it provides a useful way of assigning priorities among various tasks in preemptivescheduling.

3. Dynamic planning-based approaches:

Here, the feasible schedules are identified dynamically (at run time). It carries acertain fixed time interval and a process is executed if and only if satisfies the time constraint.

4. Dynamic best effort approaches:

These types of approaches consider deadlines instead of feasible schedules. Therefore the task is aborted if its deadline is reached. This approach is usedwidely is most of the real-time systems.

Advantages of Scheduling in Real-Time Systems:

 Meeting Timing Constraints: Scheduling ensures that real-time tasks are executed within their specified timing constraints. It guarantees that critical tasks are completed on time, preventing potential system failures or losses.

 Resource Optimization: Scheduling algorithms allocate system resources effectively, ensuring efficient utilization of processor time, memory, and other resources. This helps maximize system throughput and performance.

 Priority-Based Execution: Scheduling allows for priority-based execution, where higher-priority tasks are given precedence over lower-priority tasks. Thisensures that time-critical tasks are promptly executed, leading to improved system responsiveness and reliability.

 Predictability and Determinism: Real-time scheduling provides predictabilityand determinism in task execution. It enables developers to analyze and guarantee the worst-case execution time and response time of tasks, ensuring that critical deadlines are met.

 Control Over Task Execution: Scheduling algorithms allow developers to have fine-grained control over how tasks are executed, such as specifying taskpriorities, deadlines, and inter-task dependencies. This control facilitates the design and implementation of complex real-time systems.

Downloaded by bhagavathi 8411 (bhagavathi8411 2000@gmail.com)

Disadvantages of Scheduling in Real-Time Systems:

 Increased Complexity: Real-time scheduling introduces additional complexity to system design and implementation. Developers need to carefully analyze
task requirements, define priorities, and select suitable scheduling algorithms. This complexity can lead to increased development time and effort.

 Overhead: Scheduling introduces some overhead in terms of context switching, task prioritization, and scheduling decisions. This overhead can impact system performance, especially in cases where frequent context switches or complex scheduling algorithms are employed.

 Limited Resources: Real-time systems often operate under resourceconstrained environments. Scheduling tasks within these limitations can be challenging, as the available resources may not be sufficient to meet all timing constraints or execute all tasks simultaneously.

 Verification and Validation: Validating the correctness of real-time schedulesand ensuring that all tasks meet their deadlines require rigorous testing and verification techniques. Verifying timing constraints and guaranteeing the absence of timing errors can be a complex and time-consuming process.

 Scalability: Scheduling algorithms that work well for smaller systems may notscale effectively to larger, more complex real-time systems. As the number of tasks and system complexity increases, scheduling decisions become more challenging and may require more advanced algorithms or approaches.

This document is available on studocu

Downloaded by bhagavathi 9411 (bhagavathi9411 2000@gmail.com)

SUBJECT NAME: ADVANCED OPERATING SYSTEM

SUBJECT CODE:

UNIT: IV

UNIT-IV: HANDHELD OPERATING SYSTEM

Operating System for Handheld system: Requirements-Technology overview-Handheld operating system-Palm OS- Symbian OS- Android- Architecture of Android-Securing handheld system.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

REQUIREMENTS:

Handheld operating systems, like those on smartphones, typically have several key requirements:

1. **User Interface (UI):** Intuitive and responsive touch-based interface for user interaction.

2. **Multitasking:** Capability to run multiple applications simultaneously.

3. **Connectivity:** Support for various communication technologies such as Wi-Fi, Bluetooth, and mobile data.

4. **Security:** Robust security measures to protect user data and privacy.

5**. Power Management:** Efficient power usage and optimization to extend battery life.

6. **Application Ecosystem:** App store support, allowing users to download and install third-party applications.

7**. Compatibility:** Compatibility with a wide range of hardware devices and accessories.

8. **Updates:**Regular software updates to fix bugs, enhance features, and address security vulnerabilities.

9. **Customization:** Options for users to customize the device settings and appearance.

10. **Performance:** Smooth performance on the given hardware, with minimal lag or delays.

These requirements contribute to creating a seamless and user-friendly experience on handheld devices.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

TECHNOLOGY OVERVIEW:

Handheld systems typically refer to portable electronic devices that can be held in one's hand. Here's a brief overview of the key technologies commonly found in handheld systems:

1. **Processor:** Handheld devices feature processors (CPU) that determine their processing power. Modern handhelds often utilize energy-efficient, multi-core processors for enhanced performance.

2. **Operating System (OS)**:Mobile operating systems like Android, iOS, or others provide the software platform for handheld devices, managing applications, user interface, and device functions.

3. **Display Technology:** Handhelds employ various display technologies such as LCD or OLED for vibrant and energy-efficient screens. Touchscreen functionality is standard for user interaction.

4. **Battery Technology:** Advances in battery technology, including lithium-ion and other rechargeable batteries, are crucial for longer usage periods. Power management is a critical aspect of handheld design.

5. **Connectivity:** Handheld systems support multiple connectivity options like Wi-Fi, Bluetooth, and cellular networks. These enable internet access, data transfer, and connectivity with other devices.

6. **Storage:** Flash memory or solid-state drives provide storage for applications, media, and user data. The amount of storage varies based on the device's purpose and design.

7. **Sensors:** Accelerometers, gyroscopes, GPS, ambient light sensors, and proximity sensors enhance the device's functionality, enabling features like rotation sensing, location tracking, and adaptive screen brightness.

8**. Cameras:** Most handhelds include front and rear-facing cameras for photography, video recording, and video conferencing.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

9. **Input Methods:** Touchscreens are common, but handheld devices may also support physical buttons, virtual keyboards, stylus input, or a combination of these for user interaction.

10. **Security Features:** Biometric authentication (e.g., fingerprint sensors, facial recognition) and encryption technologies help secure handheld devices and user data.

11. **App Ecosystem:** Handhelds thrive on diverse app ecosystems. App stores provide a wide range of applications for productivity, entertainment, communication, and more.

12. **Design and Materials:** Lightweight and durable materials, along with ergonomic design, contribute to the overall user experience and portability of handheld devices.

The continual evolution of these technologies ensures that handheld systems remain versatile and capable of meeting various user needs.

HANDHELD OPERATING SYSTEM:

Handheld operating systems are available in all handheld devices like Smartphones and tablets. It is sometimes also known as a Personal Digital Assistant. The popular handheld device in today's world is Android and iOS. These operating systems need a high-processing processor and are also embedded with various types of sensors.

Some points related to Handheld operating systems are as follows:

1. Since the development of handheld computers in the 1990s, the demand for software to operate and run on these devices has increased.
2. Three major competitors have emerged in the handheld PC world with three different operating systems for these handheld PCs.
3. Out of the three companies, the first was the Palm Corporation with their PalmOS.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

4. Microsoft also released what was originally called Windows CE. Microsoft's recently released operating system for the handheld PC comes under the name of Pocket PC.
5. More recently, some companies producing handheld PCs have also started offering a handheld version of the Linux operating system on their machines.

Features of Handheld Operating System:

1. Its work is to provide real-time operations.
2. There is direct usage of interrupts.
3. Input/Output device flexibility.
4. Configurability.

Types of Handheld Operating Systems:

Types of Handheld Operating Systems are as follows:

1. Palm OS
2. Symbian OS
3. Linux OS
4. Windows
5. Android

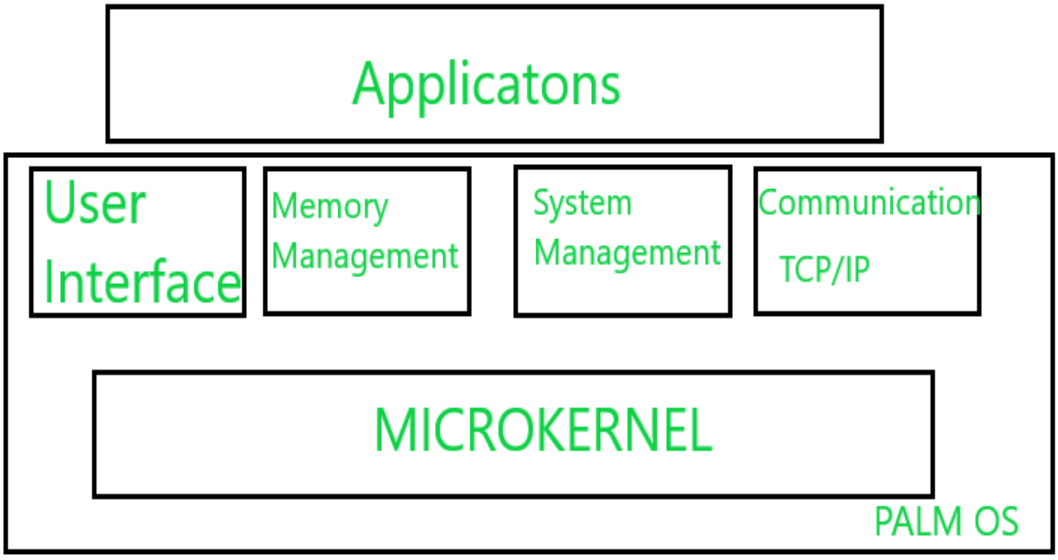
PALM OPERATING SYSTEM:

PALM OS is an operating system for personal digital assistants, designed for touchscreen. It consists of a limited number of features designed for low memory and processor usage which in turn helps in getting longer battery life.

Features of PALM OS

- Elementary memory management system.
- Provides PALM Emulator.
- Handwriting recognition is possible.
- Supports recording and playback.
- Supports C, and C++ software.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)



- The User Interface in the architecture is used for graphical input-output.
- The Memory Management section is used for maintaining databases, global variables, etc.
- System Management's job is to maintain events, calendars, dates, times, etc.
- Communication TCP/IP as the name denotes is simply used for communication.
- Microkernel is an essential tool in architecture. It is responsible for providing the mechanism needed for the proper functioning of an operating system.

Development Cycle

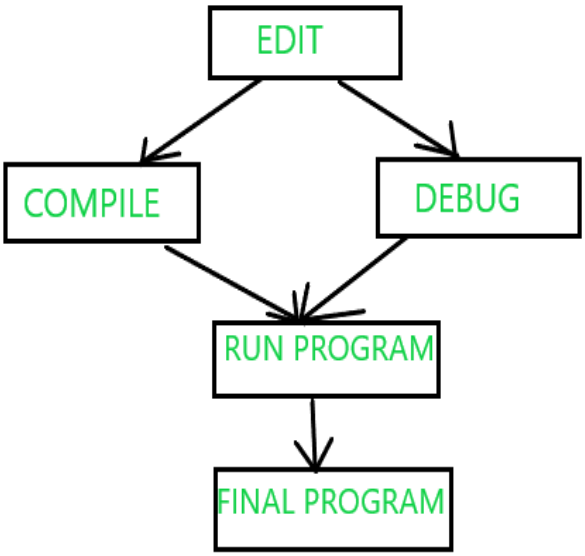
For the development of the PALM OS, these are the phases it has to go through before it can be used in the market:

- Editing the code for the operating system that is checking for errors and correcting errors.
- Compile and Debug the code to check for bugs and correct functioning of the code.
- Run the program on a mobile device or related device.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

- If all the above phases are passed, we can finally have our finished product which is the operating system for mobile devices named PALM OS.



Advantages:

- Fewer features are designed for low memory and processor usage which means longer battery life.
- No need to upgrade the operating system as it is handled automatically in PALM OS.
- More applications are available for users.
- Extended connectivity for users. Users can now connect to wide areas.

Disadvantages:

- The user cannot download applications using the external memory in PALM OS. It will be a disadvantage for users with limited internal memory.
- Systems and extended connectivity are less compared to what is offered by other operating systems.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

SYMBIAN OPERATING SYSTEM:

The Symbian OS was an operating system designed for mobile devices, primarily smartphones. It was developed by Symbian Ltd., a consortium of mobile phonemodels, including Nokia, Ericsson, and Motorola. Symbian OS was widely used in the early 2000s and offered a range of features and capabilities for mobile devices.

Features of Symbian OS:

1. Multitasking: Symbian supported multitasking, allowing users to run multiple applications simultaneously.

2. Customization: It provided a customizable interface with various themes and options for personalization**.**

3. Security: Symbian OS had a strong focus on security, incorporating features to protect user data and prevent malware.

4. Connectivity: It supported a range of connectivity options, including Bluetooth, Wi-Fi, and 3G, enhancing communication capabilities.

5. Built-in Applications: Symbian included a suite of built-in applications such as a web browser, email client, and productivity tools.

Advantages of Symbian OS:

1. Efficiency: Symbian was known for its efficient use of system resources, making it suitable for devices with limited hardware capabilities.

2. Battery Life: The OS was optimized for power efficiency, contributing to better battery life on devices running Symbian.

3. Stability: Symbian OS was generally stable, providing a reliable platform for mobile devices.

Disadvantages of Symbian OS:

1. User Interface: The user interface of Symbian was often criticized for being less intuitive and visually appealing compared to emerging competitors.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

2. App Ecosystem: Symbian struggled to compete with the rapidly expanding app ecosystems of iOS and Android, limiting the availability of third-party applications.

3. Transition Issues: The transition from older versions to newer ones faced challenges, leading to fragmentation and compatibility issues.

4. Lack of Developer Interest: As iOS and Android gained popularity, many developers shifted their focus away from Symbian, resulting in a decline in app support.

ANDROID:

Android is a mobile operating system based on a modified version of the Linux kernel and other open-source software, designed primarily for touchscreen mobile devices such as smartphones and tablets. Android is developed by a partnership of developers known as the Open Handset Alliance and commercially sponsored by Google. It was disclosed in November 2007, with the first commercial Android device, the HTC Dream, launched in September 2008.

It is free and open-source software. Its source code is Android Open Source Project (AOSP), primarily licensed under the Apache License.

Features of Android :

Below are the following unique features and **characteristics of the android operating system, such as:**

1. Near Field Communication (NFC)

Most Android devices support NFC, which allows electronic devices to interact across short distances easily. The main goal here is to create a payment option that is simpler than carrying cash or credit cards, and while the market hasn't exploded as many experts had predicted, there may be an alternative in the works, in the form of Bluetooth Low Energy (BLE).

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

2. Infrared Transmission

The Android operating system supports a built-in infrared transmitter that allows you to use your phone or tablet as a remote control.

3. Automation

The ***Tasker*** app allows control of app permissions and also automates them.

4. Wireless App Downloads

You can download apps on your PC by using the Android Market or third-party options like ***AppBrain***. Then it automatically syncs them to your Droid, and no plugging is required.

5. Storage and Battery Swap

Android phones also have unique hardware capabilities. Google's OS makes it possible to upgrade, replace, and remove your battery that no longer holds a charge. In addition, Android phones come with SD card slots for expandable storage.

6. Custom Home Screens

While it's possible to hack certain phones to customize the home screen, Android comes with this capability from the get-go. Download a third-party launcher like ***Apex, Nova***, and you can add gestures, new shortcuts, or even performance enhancements for older-model devices.

7. Widgets

Apps are versatile, but sometimes you want information at a glance instead of having to open an app and wait for it to load. Android widgets let you display just about any feature you choose on the home screen, including weather apps, music widgets, or productivity tools that helpfully remind you of upcoming meetings or approaching deadlines.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

8. Custom ROMs

Because the Android operating system is open-source, developers can twist the current OS and build their versions, which users can download and install in place of the stock OS. Some are filled with features, while others change the look and feel of a device. Chances are, if there's a feature you want, someone has already built a custom ROM for it.

Android OS versions

Google makes incremental changes to the OS with each release. This often includes security patches and performance improvements.

- **Android 1.0.** Released Sept. 23, 2008. Included a suite of Google apps, including Gmail, Maps, Calendar and YouTube.
- **Android 1.5 (Cupcake).** Released April 27, 2009. Introduced an onscreen virtual keyboard and the framework for third-party app widgets.
- **Android 1.6 (Donut).** Released Sept. 15, 2009. Introduced the ability for the OS to run on different screen sizes and resolutions; added support for CDMA networks.
- **Android 2.0 (Eclair).** Released Oct. 26, 2009. Added turn-by-turn voice navigation, real-time traffic information, pinch-to-zoom capability.
- **Android 2.2 (Froyo).** Released May 20, 2010. Added dock at the bottom of the home screen and voice actions, which allows users to tap an icon and speak a command. Also introduced support for Flash to the web browser.
- **Android 2.3 (Gingerbread).** Released Dec. 6, 2010. Introduced black and green into the UI.
- **Android 3.0 to 3.2 (Honeycomb).** Released Feb. 22, 2011. This release was exclusive to tablets and introduced a blue, space-themed holographic design.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

- **Android 4.0 (Ice Cream Sandwich).** Released Oct. 18, 2011. Introduced a unified UI to both tablets and smartphones; emphasized swiping as a navigational method.
- **Android 4.1 to 4.3 (Jelly Bean).** Released July 9, 2012, Nov. 13, 2012, and July 24, 2013, respectively. Introduced Google Now, a day planner service. Added interactive notifications and improved voice search system.
- **Android 4.4 (KitKat).** Released Oct. 31, 2013. Introduced lighter colors into the UI, along with a transparent status bar and white icons.
- **Android 5.0 (Lollipop).** Released Nov. 12, 2014. Incorporated a card-based appearance in the design with elements such as notifications and Recent Apps list. Introduced hands-free voice control with the spoken "OK, Google" command.
- **Android 6.0 (Marshmallow).** Released Oct. 5, 2015. This release marked Google's adoption of an annual release schedule. Introduced more granular app permissions and support for USB-C and fingerprint readers.
- **Android 7.0 and 7.1 (Nougat).** Released Aug. 22, 2016 and Oct. 4, 2016, respectively. Introduced a native split-screen mode and the ability to bundle notifications by app.
- **Android 8.0 and 8.1 (Oreo).** Released Aug. 21, 2017 and Dec. 5, 2017, respectively. These versions introduced a native picture-in-picture (PIP) mode and the ability to snooze notifications. Oreo was the first version to incorporate Project Treble, an effort by OEMs to provide more standardized software updates.
- **Android 9.0 (Pie).** Released Aug. 6, 2018. This version replaced Back, Home and Overview buttons for a multifunctional Home button and a smaller Back button. Introduced productivity features, including suggested replies for messages and brightness management capabilities.
- **Android 10 (Android Q).** Released Sept. 3, 2019. Abandoned the Back button in favor of a swipe-based approach to navigation. Introduced a dark theme and Focus Mode, which enables users to limit distractions from certain apps.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

- **Android 11 (Red Velvet Cake).** Released Sept. 8, 2020. Added built-in screen recording. Created a single location to view and respond to conversations across multiple messaging apps. This version also updated the chat bubbles so users can pin conversations to the top of apps and screens.
- **Android 12 (Snow Cone).** Released Oct. 4, 2021. Added customization options for the user interface. The conversation widget let users store preferred contacts on their home screens. Added more privacy options, including sharing when apps access information such as camera, photos or microphone.
- **Android 12L.** Released March 7, 2022.The L stands for larger screens. This update aimed to improve the user interface and optimize for the larger screen of a tablet, foldable or Chromebook. This update added a dual-panel notification center for tablets and foldables.
- **Android 13 (Tiramisu).** Released Aug. 15, 2022. Included more customizable options including color, theme, language and music. Security updates included control over information apps can access, notification permission required for all apps and clearing of personal information on clipboard. This update enables multitasking by sharing of messages, chats, links and photos across multiple Android devices -- including phones, tablets and Chromebooks.
- **Android 14 (Upside Down Cake).** Released Oct. 4, 2023. Included more customization options for the lock screen and wallpapers. OS efficiency was improved to decrease the strain on a phone’s battery. For accessibility, Google added larger scalable fonts for vision-impaired users and camera flashes to give hearing-impaired users another visual cue when they get a notification. Security updates include notifications of changes in data-sharing policies for third-party applications, enhanced PIN security features and better support for passkey authentication across more applications.

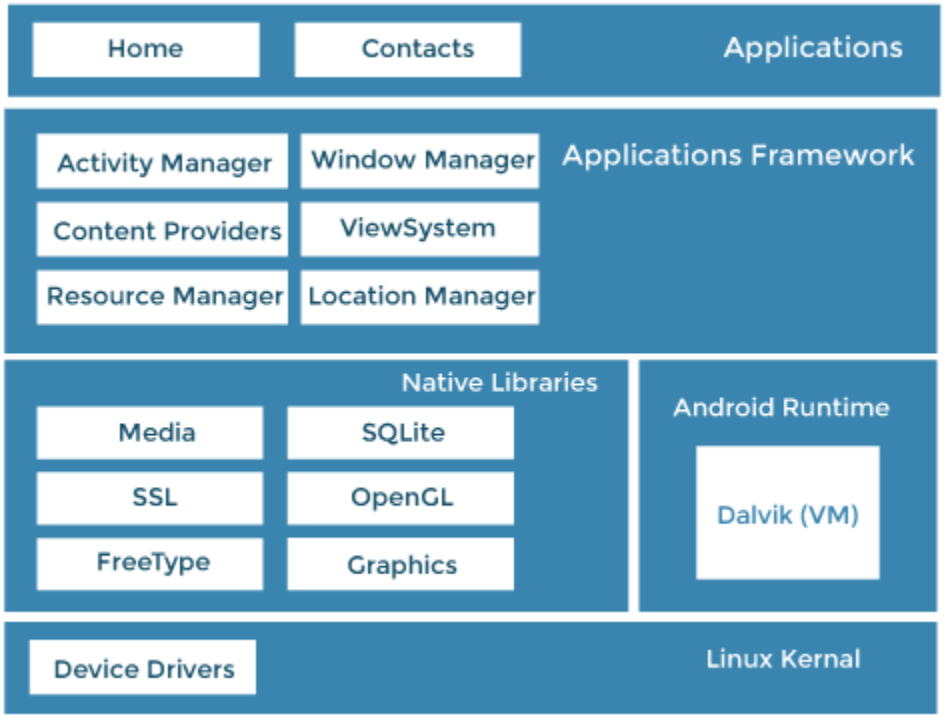
Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

ANDROID ARCHITECTURE

The android architecture contains a different number of components to support any android device needs. Android software contains an open-source Linux Kernel with many C/C++ libraries exposed through application framework services.

Among all the components, Linux Kernel provides the main operating system functions to Smartphone and Dalvik Virtual Machine (DVM) to provide a platform for running an android application. An android operating system is a stack of software components roughly divided into five sections and four main layers, as shown in the below architecture diagram.

- Applications
- Application Framework
- Android Runtime
- Platform Libraries
- Linux Kernel



This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

1. Applications

An application is the top layer of the android architecture. The pre-installed applications like camera, gallery, home, contacts, etc., and third-party applications downloaded from the play store like games, chat applications, etc., will be installed on this layer.

It runs within the Android run time with the help of the classes and services provided by the application framework.

2. Application framework

Application Framework provides several important classes used to create an Android application. It provides a generic abstraction for hardware access and helps in managing the user interface with application resources. Generally, it provides the services with the help of which we can create a particular class and make that class helpful for the Applications creation.

It includes different types of services, such as activity manager, notification manager, view system, package manager etc., which are helpful for the development of our application according to the prerequisite.

The Application Framework layer provides many higher-level services to applications in the form of Java classes. Application developers are allowed to make use of these services in their applications. The Android framework includes the following key services:

- **Activity Manager:** Controls all aspects of the application lifecycle and activity stack.
- **Content Providers:** Allows applications to publish and share data with other applications.
- **Resource Manager:** Provides access to non-code embedded resources such as strings, colour settings and user interface layouts.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

- **Notifications Manager:** Allows applications to display alerts and notifications to the user.
- **View System:** An extensible set of views used to create application user interfaces.

3. Application runtime

Android Runtime environment contains components like core libraries and the Dalvik virtual machine (DVM). It provides the base for the application framework and powers our application with the help of the core libraries.

Like **Java Virtual Machine** (JVM), **Dalvik Virtual Machine** (DVM) is a register-based virtual machine designed and optimized for Android to ensure that a device can run multiple instances efficiently.

It depends on the layer Linux kernel for threading and low-level memory management. The core libraries enable us to implement android applications using the standard **JAVA** or **Kotlin** programming languages.

4. Platform libraries

The Platform Libraries include various C/C++ core libraries and Java-based libraries such as Media, Graphics, Surface Manager, OpenGL, etc., to support Android development.

- **app:** Provides access to the application model and is the cornerstone of all Android applications.
- **content:** Facilitates content access, publishing and messaging between applications and application components.
- **database:** Used to access data published by content providers and includes SQLite database, management classes.
- **OpenGL:** A Java interface to the OpenGL ES 3D graphics rendering API.
- **os:** Provides applications with access to standard operating system services, including messages, system services and inter-process communication.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

- **text:** Used to render and manipulate text on a device display.
- **view:** The fundamental building blocks of application user interfaces.
- **widget:** A rich collection of pre-built user interface components such as buttons, labels, list views, layout managers, radio buttons etc.
- **WebKit:** A set of classes intended to allow web-browsing capabilities to be built into applications.
- **media:** Media library provides support to play and record an audio and video format.
- **surface manager:** It is responsible for managing access to the display subsystem.
- **SQLite:** It provides database support, and FreeType provides font support.
- **SSL:** Secure Sockets Layer is a security technology to establish an encrypted link between a web server and a web browser.

5. Linux Kernel

Linux Kernel is the heart of the android architecture. It manages all the available drivers such as display, camera, Bluetooth, audio, memory, etc., required during the runtime.

The Linux Kernel will provide an abstraction layer between the device hardware and the other android architecture components. It is responsible for the management of memory, power, devices etc. The features of the Linux kernel are:

- **Security:** The Linux kernel handles the security between the application and the system.
- **Memory Management:** It efficiently handles memory management, thereby providing the freedom to develop our apps.
- **Process Management:** It manages the process well, allocates resources to processes whenever they need them.
- **Network Stack:** It effectively handles network communication.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

- **Driver Model:** It ensures that the application works properly on the device and hardware manufacturers responsible for building their drivers into the Linux build.

SECURING HANDHELD SYSTEM

To secure a handheld system:

1. **Device Lock:** Set a strong PIN, password, or biometric authentication to prevent unauthorized access.

2. **Software Updates:** Keep the operating system and apps up-to-date to patch security vulnerabilities.

3**. App Permissions:** Review and limit app permissions to only essential functions to minimize potential security risks.

4. **Install from Trusted Sources:** Download apps only from official app stores to reduce the risk of malware.

5**. Encrypt Data:** Enable device encryption to protect your data in case the device is lost or stolen.

6. **Remote Tracking and Wiping:** Use built-in features or third-party apps to remotely track, lock, or wipe your device in case it's lost or stolen.

7**. Network Security:** Avoid connecting to public Wi-Fi networks, and use a VPN for added security when necessary.

8. **Backup Regularly:** Backup your data regularly to a secure location to prevent data loss in case of device compromise.

9**. Bluetooth and NFC:** Disable these features when not in use to prevent unauthorized access or data interception.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

10. **Awareness:** Be cautious of phishing attempts and only click on links or download attachments from trusted sources.

Implementing these measures helps enhance the overall security of your handheld device.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

SHANMUGA INDUSTRIES ARTS & SCIENCE COLLEGE, T.V.MALAI.

PG AND RESEARCH DEPARTMENT OF COMPUTER SCIENCE

CLASS: I M.SC CS

SUBJECT NAME: ADVANCED OPERATING SYSTEM

SUBJECT CODE: Type your text

SYLLABUS

UNIT-V

CASE STUDIES

Linux system: Introduction – Memory management – Process scheduling – Scheduling policy- Managing I/O devices- Accessing files- IOS: Architecture and SDK framework- Media layer –Service layer- Core OS layer- file system.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

LINUX SYSTEM

INTRODUCTION:

The Linux Operating System is a type of operating system that is similar to Unix, and it is built upon the Linux Kernel. The Linux Kernel is like the brain of the operating system because it manages how the computer interacts with its hardware and resources. It makes sure everything works smoothly and efficiently. But the Linux Kernel alone is not enough to make a complete operating system. To create a full and functional system, the Linux Kernel is combined with a collection of software packages and utilities, which are together called Linux distributions. These distributions make the Linux Operating System ready for users to run their applications and perform tasks on their computers securely and effectively. Linux distributions come in different flavors, each tailored to suit the specific needs and preferences of users.

History of Linux

Linux is a powerful and flexible family of operating systems that are free to use and share. It was created by a person named Linus Torvalds in 1991. What's cool is that anyone can see how the system works because its source code is open for everyone to explore and modify. This openness encourages people from all over the world to work together and make Linux better and better. Since its beginning, Linux has grown into a stable and safe system used in many different things, like computers, smartphones, and big supercomputers. It's known for being efficient, meaning it can do a lot of tasks quickly, and it's also cost-effective, which means it doesn't cost a lot to use. Lots of people love Linux, and they're part of a big community where they share

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

ideas and help each other out. As technology keeps moving forward, Linux will keep evolving and staying important in the world of computers.

Linux Distribution

Linux distribution is an operating system that is made up of a collection of software based on Linux kernel or you can say distribution contains the Linux kernel and supporting libraries and software. And you can get Linux based operating system by downloading one of the Linux distributions and these distributions are available for different types of devices like embedded devices, personal computers, etc. Around **600 + Linux Distributions** are available and some of the popular Linux distributions are:

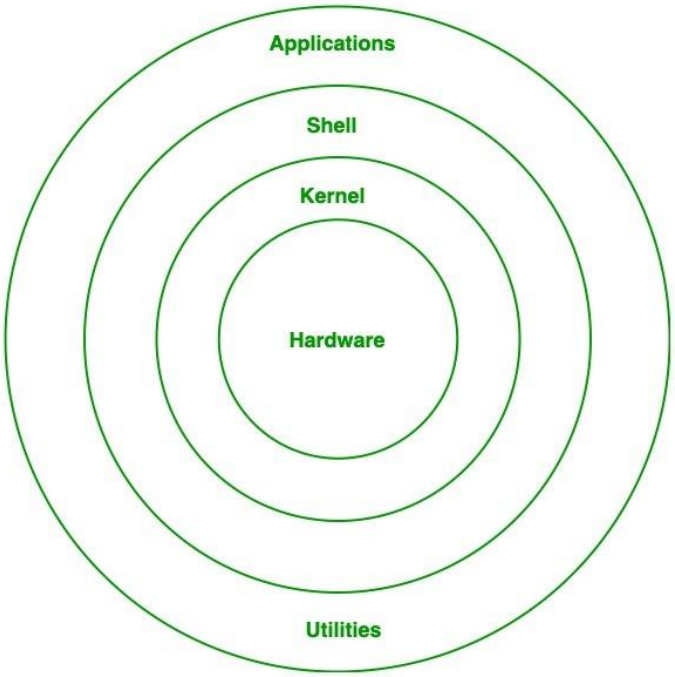
- MX Linux
- Manjaro
- Linux Mint
- elementary
- Ubuntu
- Debian
- Solus
- Fedora
- openSUSE
- Deepin

Architecture of Linux

Linux architecture has the following components:

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)



Linux Architecture

1. **Kernel:** Kernel is the core of the Linux based operating system. It virtualizes the common hardware resources of the computer to provide each process with its virtual resources. This makes the process seem as if it is the sole process running on the machine. The kernel is also responsible for preventing and mitigating conflicts between different processes. Different types of the kernel are:
   - Monolithic Kernel
   - Hybrid kernels
   - Exo kernels
   - Micro kernels
2. **System Library:**Linux uses system libraries, also known as shared libraries, to implement various functionalities of the operating system. These libraries contain pre-written code that applications can use to perform specific tasks. By using these libraries, developers can save time and effort, as they don't need to write the same code repeatedly. System libraries act as an interface between

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

applications and the kernel, providing a standardized and efficient way for applications to interact with the underlying system.

3. **Shell:**The shell is the user interface of the Linux Operating System. It allows users to interact with the system by entering commands, which the shell interprets and executes. The shell serves as a bridge between the user and the kernel, forwarding the user's requests to the kernel for processing. It provides a convenient way for users to perform various tasks, such as running programs, managing files, and configuring the system.
4. **Hardware Layer:** The hardware layer encompasses all the physical components of the computer, such as RAM (Random Access Memory), HDD (Hard Disk Drive), CPU (Central Processing Unit), and input/output devices. This layer is responsible for interacting with the Linux Operating System and providing the necessary resources for the system and applications to function properly. The Linux kernel and system libraries enable communication and control over these hardware components, ensuring that they work harmoniously together.
5. **System Utility:** System utilities are essential tools and programs provided by the Linux Operating System to manage and configure various aspects of the system. These utilities perform tasks such as installing software, configuring network settings, monitoring system performance, managing users and permissions, and much more. System utilities simplify system administration tasks, making it easier for users to maintain their Linux systems efficiently.

Advantages of Linux

- The main advantage of Linux is it is an open-source operating system. This means the source code is easily available for everyone

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

and you are allowed to contribute, modify and distribute the code to anyone without any permissions.

- In terms of security, Linux is more secure than any other operating system. It does not mean that Linux is 100 percent secure, it has some malware for it but is less vulnerable than any other operating system. So, it does not require any anti-virus software.
- The software updates in Linux are easy and frequent.
- Various Linux distributions are available so that you can use them according to your requirements or according to your taste.
- Linux is freely available to use on the internet.
- It has large community support.
- It provides high stability. It rarely slows down or freezes and there is no need to reboot it after a short time.
- It maintains the privacy of the user.
- The performance of the Linux system is much higher than other operating systems. It allows a large number of people to work at the same time and it handles them efficiently.
- It is network friendly.
- The flexibility of Linux is high. There is no need to install a complete Linux suite; you are allowed to install only the required components.
- Linux is compatible with a large number of file formats.
- It is fast and easy to install from the web. It can also install it on any hardware even on your old computer system.
- It performs all tasks properly even if it has limited space on the hard disk.

Disadvantages of Linux

- It is not very user-friendly. So, it may be confusing for beginners.

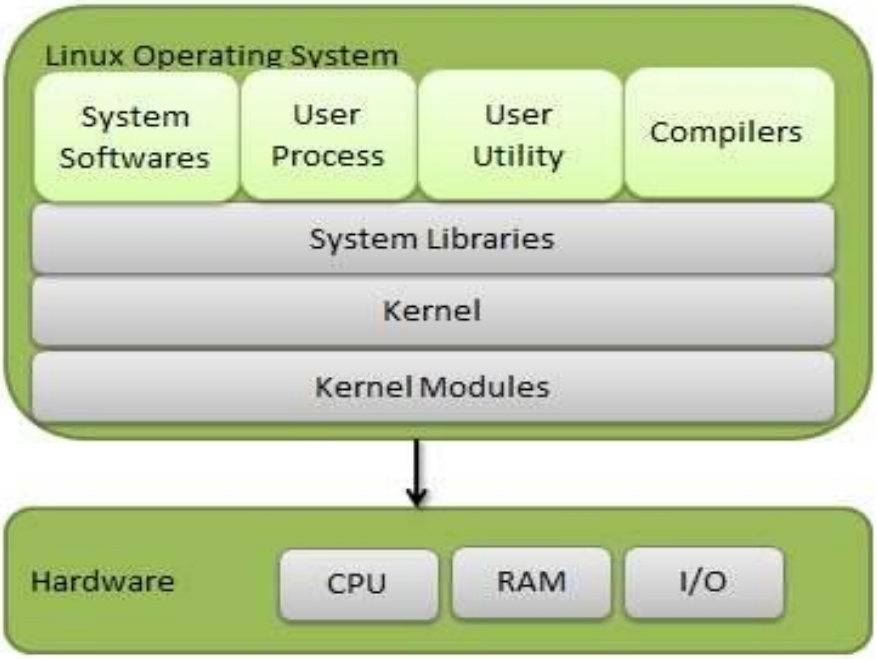
Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

- It has small peripheral hardware drivers as compared to windows.

Components of Linux System

Linux Operating System has primarily three components

- **Kernel** – Kernel is the core part of Linux. It is responsible for all major activities of this operating system. It consists of various modules and it interacts directly with the underlying hardware. Kernel provides the required abstraction to hide low level hardware details to system or application programs.
- **System Library** – System libraries are special functions or programs using which application programs or system utilities accesses Kernel's features. These libraries implement most of the functionalities of the operating system and do not requires kernel module's code access rights.
- **System Utility** – System Utility programs are responsible to do specialized, individual level tasks.



This document is available on studocu

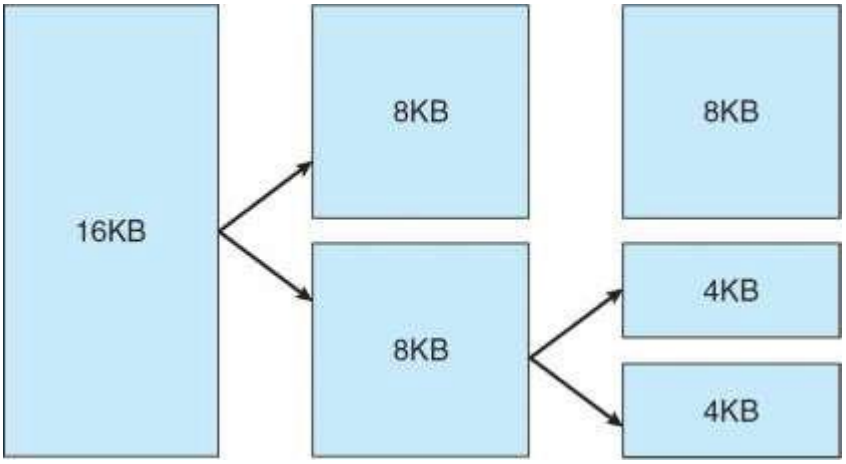
Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

MEMORY MANAGEMENT

Memory management includes allocating and freeing physical memory – pages, groups of pages, and small blocks of main memory and handling virtual memory that is memory mapped into the address space of running processes.

Management of Physical Memory – Buddy System

In this section we learn what the buddy system is and how it is used for allocating pages. Adjacent units of allocatable memory are paired together. Each allocatable memory region has an adjacent partner called a buddy. Whenever two allocated partner regions (adjacent regions) are freed up, they are combined to form a larger region called a buddy heap. The larger region also has a partner with which it can combine to form a still larger free region. If there is a request for a small region and if a region of that size is not available, a larger region is split into two partners. This splitting can happen till the region goes down to the size of a page. That is, the smallest size that can be allocated is a single page.



Splitting of Memory in a Buddy Heap

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

It shows how memory is split in a buddy heap when there is a request for a smaller memory space. Suppose there is a request for a memory space of 4KB and the region that is available is of size 16 KB. This large region is first split into two partners of 8 KB size each. One of the two 8 KB regions is split into two 4 KB regions. One of the two 4 KB regions is allocated to the requesting process. Now, there is 4 KB free region and a 8 KB free region.

Virtual Memory

The VM system maintains the address space accessible to each process. It creates pages of virtual memory on demand, and manages the loading of those pages from disk or their swapping back out to disk as required.

There are several types of virtual memory regions. The regions are usually backed by a file or by nothing (*demand-zero* memory). For example, a region may be taken from a file's contents, if it is a text region. When a region is backed by nothing, and when that region is accessed, a page filled with zeros is given. For regions having uninitialized data, the regions may not be backed by anything and may be filled with zeros.The VM manager maintains two separate views of a process's address space. One is the logical view describing instructions concerning the layout of the address space. The address space consists of a set of nonoverlapping regions, each region representing a continuous, page-aligned subset of the address space. That is, the logical view looks at a process as set of regions such as the text region, data region and stack region. The other view is the physical view of each address space. This is stored in the hardware page tables for the process. Page table entries have the current location of each page of virtual memory – whether it is in disk or

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

in physical memory. This view looks at a process as set of pages. The backing store describes from where the pages for a region come from.

A virtual memory region is also defined by the region's reaction to writes. If the region is a shared region, multiple processes can share the region at the same time. If the region is a private region, copy-on-write is set for the region. In this case, whenever one of the processes that is sharing the region tries to write to the region, a copy of the region is made for the writing process.

The kernel creates a new virtual address space in two situations. One is when a process runs a new program with the exec system call and the other is upon creation of a new process by the fork system call. When the fork system call s executed, a new process is created, but, a new program is not run.

On executing a new program using the exec system call, the process is given a new, completely empty virtual-address space. The virtual address space that was existing before the call is totally erased and is empty now. The program-loading routines populate the address space with virtual-memory regions.

Creating a new process with fork involves creating a complete copy of the existing process's virtual address space. The kernel copies the parent process's virtual memory address descriptors, then creates a new set of page tables for the child. The parent's page tables are copied directly into the child's, with the reference count of each page covered being incremented. After the fork, the parent and child share the same physical

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

pages of memory in their address spaces. If the virtual memory region is private, the pages for the regions are marked as read-only and copy-on-write is set.

Virtual Memory – Swapping and Paging

The virtual memory paging system relocates pages of memory from physical memory out to disk when the memory is needed for something else. When there is no enough space in the main memory, some of the pages have to be moved to the swap space and may be moved into the main memory later when required. In Linux, there is no whole-process swapping. Only individual pages are moved.

In Linux, the virtual memory paging system can be divided into two sections: One is the pageout-policy algorithm that decides which pages to write out to disk, and when. In Linux, the least frequently used (LFU) principle is used. The second is the paging mechanism that actually carries out the transfer, and pages data back into physical memory as needed.

PROCESS SCHEDULING

Process scheduling is one of the most important aspects or roles of any operating system.

- Process Scheduling is an important activity performed by the *process manager* of the respective operating system.
- **Scheduling in Linux deals with the removal of the current process from the CPU and selecting another process for execution**.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

Let us learn more about scheduling strategies (scheduling algorithms) used in Linux operating systems.

Similar to the other UNIX-based operating systems, in LINUX a Process Scheduler deals with process scheduling.

- Process Scheduler chooses a process to be executed.
- Process scheduler also decides for how long the chosen process is to be executed.

Hence, Scheduling Algorithms is nothing but a kind of strategy that helps the process scheduler in deciding the process to be executed.

Process scheduling in a Linux system involves managing the execution of multiple processes on the CPU. The Linux scheduler is responsible for determining which process should run at any given time. Here's a detailed overview:

1. **Scheduler Components:**

- **Completely Fair Scheduler (CFS):** The primary scheduler in Linux, designed to provide fair CPU allocation to processes based on their priority and resource requirements.
- **Real-Time Scheduler:** Supports real-time processes with strict timing requirements.

2. Process Priority:

- Each process is assigned a priority value, often referred to as "niceness."

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

- Lower niceness values indicate higher priority. Real-time processes have their own priority scheme.

3. Time Slices (Quantum):

- The scheduler divides time into small intervals called time slices or quantum.
- Each process is allocated a time slice during which it can execute.

4. Process States:

- **Running:** The process is currently using the CPU.
- **Ready:** The process is ready to run but waiting for its turn.
- **Blocked (or Waiting):** The process is waiting for an event, like I/O completion.

5. Scheduling Policies:

- **Time-sharing (CFS):** Provides fair distribution of CPU time among all processes.
- **Real-Time (RT):** Ensures that real-time processes meet their deadlines.

6**. CFS Algorithm:**

- Assigns each process a virtual runtime.
- The process with the smallest virtual runtime is selected to run next.

7. Process Wake-up:

- When a process becomes ready, it is placed in the run queue.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

- The scheduler then determines the order in which processes will run based on their priorities and virtual runtimes.

8. Priority Adjustment:

- CFS adjusts priorities dynamically to ensure fairness.
- If a process uses less than its allocated time slice, its priority is increased; if it uses more, its priority is decreased.

9. Nice Values:

- The nice command allows users to influence process priority.
- Lower nice values (higher priority) can be set to important processes.

10. Real-Time Scheduler:

- Supports tasks with strict timing requirements.
- Uses priority levels and predefined policies for scheduling.

11. Scheduler Classes:

- Different scheduler classes cater to various workloads (e.g., SCHED_NORMAL for time-sharing, SCHED_FIFO for real-time).

12. Affinity and Migration:

- Affinity controls on which CPU a process can run.
- Migration refers to moving processes between CPUs to balance load.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

In summary, Linux process scheduling involves managing priorities, time slices, and various policies to ensure fair CPU allocation among processes, with additional support for real-time tasks. The scheduler continuously adjusts priorities based on a process's behavior to maintain balance and efficiency.

SCHEDULING POLICY

The set of rules used to determine when and how to select a new process to run is called *scheduling policy* .

Linux scheduling is based on the *time sharing* technique: several processes run in “time multiplexing” because the CPU time is divided into *slices*, one for each runnable process. Of course, a single processor can run only one process at any given instant. If a currently running process is not terminated when its time slice or *quantum* expires, a process switch may take place. Time sharing relies on timer interrupts and is thus transparent to processes. No additional code needs to be inserted in the programs to ensure CPU time sharing.

The scheduling policy is also based on ranking processes according to their priority. Complicated algorithms are sometimes used to derive the current priority of a process, but the end result is the same: each process is associated with a value that tells the scheduler how appropriate it is to let the process run on a CPU.

In Linux, process priority is dynamic. The scheduler keeps track of what processes are doing and adjusts their priorities periodically; in this way, processes that have been denied the use of a CPU for a long time interval . In the LINUX operating system, we have mainly two types of

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

processes namely - Real-time Process and Normal Process. Let us learn more about them in detail.

Realtime Process

Real-time processes are processes that cannot be delayed in any situation. Real-time processes are referred to as **urgent processes**.

There are mainly two types of real-time processes in LINUX namely:

- SCHED_FIFO
- SCHED_RR.

A real-time process will try to seize all the other working processes having lesser priority.

For example, A *migration process* that is responsible for the distribution of the processes across the CPU is a real-time process. Let us learn about different scheduling policies used to deal with real-time processes briefly.

SCHED_FIFO
FIFO in SCHED_FIFO means First In First Out. Hence,
the SCHED_FIFO policy schedules the processes according to the arrival time of the process.

SCHED_RR
RR in SCHED_RR means **Round Robin**. The SCHED_RR policy schedules the processes by giving them a fixed amount of time for execution. **This fixed time is known as time quantum**.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

Normal Process

Normal Processes are the opposite of real-time processes. Normal processes will execute or stop according to the time assigned by the process scheduler. Hence, a normal process can suffer some delay if the CPU is busy executing other high-priority processes. Let us learn about different scheduling policies used to deal with the normal processes in detail.

Normal (SCHED_NORMAL or SCHED_OTHER)
SCHED_NORMAL / SCHED_OTHER is the default or standard scheduling policy used in the LINUX operating system. A time-sharing mechanism is used in the normal policy. A time-sharing mechanism means assigning some specific amount of time to a process for its execution. **Normal policy deals with all the threads of processes that do not need any real-time mechanism**.

Batch (SCHED_BATCH)
As the name suggests, the SCHED_BATCH policy is used for executing a batch of processes. This policy is somewhat similar to the Normal policy. SCHED_BATCH policy deals with the non-interactive processes that are useful in optimizing the CPU throughput
time. SCHED_BATCH scheduling policy is used for a group of processes having priority: 0.

MANAGING I/O DEVICES

In Linux, managing input and output devices involves handling various hardware peripherals like keyboards, mice, monitors, printers, etc.

The Linux device-oriented file system accesses disk storage through two caches:

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

Data is cached in the page cache, which is unified with the virtual memory system

Metadata is cached in the buffer cache, a separate cache indexed by the physical disk block.

Linux splits all devices into three classes:

- **Block devices** allow random access to completely independent, fixed size blocks of data
- **Character devices** include most other devices; they don't need to support the functionality of regular files
- **Network devices** are interfaced via the kernel's networking subsystem.

BLOCK DEVICES

Provide the main interface to all disk devices in a system ,

The block buffer cache serves two main purposes:

- It acts as a pool of buffers for active I/O
- It serves as a cache for completed I/O

The request manager manages the reading and writing of buffer contents to and from a block device driver .

Kernel 2.6 introduced Completely Fair Queueing (CFQ)

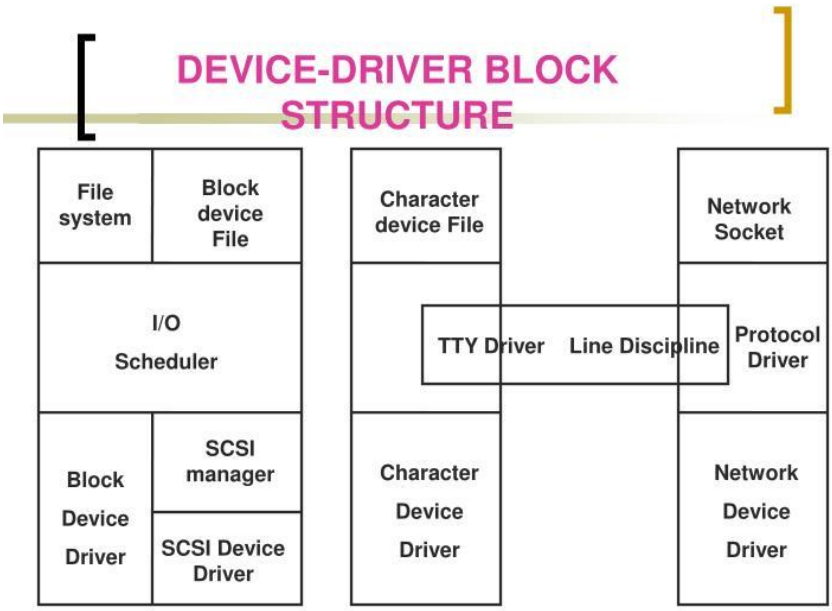
Now the default scheduler Fundamentally different from elevator algorithms

Maintains set of lists, one for each process by default

Uses C-SCAN algorithm, with round robin between all outstanding I/O from all processes

Four blocks from each process put on at once

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)



CHARACTER DEVICE

A device driver which does not offer random access to fixed blocks of data .

A character device driver must register a set of functions which implement the driver's various file I/O operations .

The kernel performs almost no preprocessing of a file read or write request to a character device, but simply passes on the request to the device .

The main exception to this rule is the special subset of character device drivers which implement terminal devices, for which the kernel maintains a standard interface

Line discipline is an interpreter for the information from the terminal device .

The most common line discipline is tty discipline, which glues the terminal's data stream onto standard input and output streams of

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

user’s running processes, allowing processes to communicate directly with the user’s terminal .

Several processes may be running simultaneously, the line discipline responsible for attaching and detaching terminal’s input and output from various processes connected to it as processes are suspended or awakened by user.

Other line disciplines also are implemented have nothing to do with I/O to user process – i.e. PPP and SLIP networking protocols.

INTERPROCESS COMMUNICATION

Like UNIX, Linux informs processes that an event has occurred via signals .

There is a limited number of signals, and they cannot carry information: Only the fact that a signal occurred is available to a process .

The Linux kernel does not use signals to communicate with processes with are running in kernel mode, rather, communication within the kernel is accomplished via scheduling states and **wait_queue** structures. Also implements System V Unix semaphores .

- Process can wait for a signal or a semaphore .
- Semaphores scale better .
- Operations on multiple semaphores can be atomic

Passing Data Between Processes

The pipe mechanism allows a child process to inherit a communication channel to its parent, data written to one end of the pipe can be read a the other.

Shared memory offers an extremely fast way of communicating; any data written by one process to a shared memory region can be read

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

immediately by any other process that has mapped that region into its address space.

To obtain synchronization, however, shared memory must be used in conjunction with another Interprocesscommunication mechanism.

NETWORK STRUCTURE

Networking is a key area of functionality for Linux.

It supports the standard Internet protocols for UNIX to UNIX communications.

It also implements protocols native to non-UNIX operating systems, in particular, protocols used on PC networks, such as Appletalk and IPX.

Internally, networking in the Linux kernel is implemented by three layers of software:

- The socket interface
- Protocol drivers
- Network device drivers

Most important set of protocols in the Linux networking system is the internet protocol suite.

It implements routing between different hosts anywhere on the network .

On top of the routing protocol are built the UDP, TCP and ICMP protocols.

Packets also pass to firewall management for filtering based on firewall chains of rules

Here are key aspects of managing these devices:

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

1. Device Drivers:

- Linux uses device drivers to communicate with hardware. Drivers are modules that allow the kernel to interact with specific hardware components.
- Kernel modules are loaded dynamically, and the Linux kernel often comes with a wide range of built-in drivers.

2. Device Files:

- Devices in Linux are represented as files in the /dev directory. Each device has a corresponding device file, and interacting with these files allows applications and users to communicate with the hardware.

3. Input Devices:

- Input devices like keyboards and mice are managed by the kernel's input subsystem.
- Configuration files, such as those found in /etc/X11/ for X.Org, can be used to customize input device settings.

4. Output Devices:

- Monitors and printers are examples of output devices. The X.Org server, Wayland, or other display servers handle graphical output.
- Printers are managed through the Common Unix Printing System (CUPS), and configuration files are typically located in /etc/cups/.

5. udev:

- udev is a Linux subsystem responsible for managing device nodes in the /dev directory dynamically.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

- Rules defined in /etc/udev/rules.d/ can be used to customize device permissions and naming.

6. Monitoring and Configuration:

- Tools like lsusb, lspci, and lsblk can be used to list connected USB devices, PCI devices, and block devices, respectively.
- Configuration files and tools for managing devices can vary based on the Linux distribution and desktop environment in use.

7. X.Org and Wayland:

- X.Org and Wayland are display servers that handle graphics and user input. Configuration files, such as xorg.conf, are used to customize display settings under X.Org.

8. Console Devices:

- The Linux console itself is a device, and configurations for consoles can be found in files like /etc/console-setup/.

Managing input and output devices in Linux involves understanding the specific tools and configurations relevant to the distribution and display system being used.

ACCESSING FILES

In Linux, most of the operations are performed on files. And to handle these files Linux has directories also known as folders which are maintained in a tree-like structure. Though, these directories are also a type of file themselves. Linux has 3 types of files:

1. **Regular Files:** It is the common file type in Linux. it includes files like – text files, images, binary files, etc. Such files can be created using the touch command. They consist of the majority of files in the

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

Linux/UNIX system. The regular file contains ASCII or Human Readable text, executable program binaries, program data and much more.

2. **Directories:** Windows call these directories as folders. These are the files that store the list of file names and the related information. The root directory(/) is the base of the system, /home/ is the default location for user's home directories, /bin for Essential User Binaries, /boot – Static Boot Files, etc. We could create new directories with mkdir command.
3. **Special Files:** Represents a real physical device such as a printer which is used for IO operations. Device or special files are used for device Input/Output(I/O) on UNIX and Linux systems. You can see them in a file system like an ordinary directory or file.

In Unix systems, there are two types of special files for each device, i.e. character special files and block special files. For more details, read the article **Unix file system**.

1. Files Listing

To perform Files listings or to list files and directories ls command is used

$ls

All your files and directories in the current directory would be listed and each type of file would be displayed with a different color. Like in the output directories are displayed with dark blue color.

$ls –l

It returns the detailed listing of the files and directories in the current directory. The command gives os the owner of the file and even which

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

file could be managed by which user or group and which user/group has the right to access or execute which file.

2. Creating Files

The touch command can be used to create a new file. It will create and open a new blank file if the file with a filename does not exist. And in case the file already exists then the file will not be affected.

$touch filename

3. Displaying File Contents

The cat command can be used to display the contents of a file. This command will display the contents of the 'filename' file. And if the output is very large then we could use more or less to fit the output on the terminal screen otherwise the content of the whole file is displayed at once.

$cat filename

4. Copying a File

The cp command could be used to create the copy of a file. It will create the new file in destination with the same name and content as that of the file 'filename'.

$cp source/filename destination/

5.Moving a File

The mv command could be used to move a file from source to destination. It will remove the file filename from the source folder and

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

would be creating a file with the same name and content in the destination folder.

$mv source/filename destination/

6. Renaming a File

The mv command could be used to rename a file. It will rename the filename to new_filename or in other words, it will remove the filename file and would be creating a new file with the new_filename with the same content and name as that of the filename file.

$mv filename new_filename

7. Deleting a File

The rm command could be used to delete a file. It will remove the filename file from the directory.

$rm filename

IOS

iOS is a mobile operating system developed by Apple Inc. exclusively for its smartphones. It was unveiled in January 2007 for the first-generation iPhone, launched in June 2007.

It is the operating system that powers many of the company's mobile devices, including the iPhone. It was also used on iPads (introduced: January 2010; availability: April 2010) until iPadOS was introduced in 2019, and on the iPod Touch devices, which were discontinued in mid-2022.It is the world's second-most widely installed mobile operating system, after Android.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

It is the basis for three other operating systems made by Apple:

- iPadOS,
- tvOS
- watchOS.

As of December 2023, Apple's App Store contains more than 3.8 million iOS applications.These mobile apps have collectively been downloaded more than 130 billion times.

iOS is based on macOS. Like macOS, it includes components of the Mach microkernel and FreeBSD.It is a Unix-like operating system. Although some parts of iOS are open source under the Apple Public Source License and other licenses, iOS is proprietary software.

Major versions of iOS are released annually. The current stable version, iOS 17, was released to the public on September 18, 2023.

Features of iOS operating System:

Let us discuss some features of the iOS operating system-

1. Highly Securer than other operating systems.
2. iOS provides multitasking features like while working in one application we can switch to another application easily.
3. iOS's user interface includes multiple gestures like swipe, tap, pinch, Reverse pinch.
4. iBooks, iStore, iTunes, Game Center, and Email are user-friendly.
5. It provides Safari as a default Web Browser.
6. It has a powerful API and a Camera.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

7. It has deep hardware and software integration

Applications of IOS Operating System:

Here are some applications of the iOS operating system-

1. iOS Operating System is the Commercial Operating system of Apple Inc. and is popular for its security.
2. iOS operating system comes with pre-installed apps which were developed by Apple like Mail, Map, TV, Music, Wallet, Health, and Many More.
3. Swift Programming language is used for Developing Apps that would run on IOS Operating System.
4. In iOS Operating System we can perform Multitask like Chatting along with Surfing on the Internet.

Advantages of IOS Operating System:

The iOS operating system has some advantages over other operating systems available in the market especially the Android operating system. Here are some of them-

1. More secure than other operating systems.
2. Excellent UI and fluid responsive
3. Suits best for Business and Professionals
4. Generate Less Heat as compared to Android.

Disadvantages of IOS Operating System:

Let us have a look at some disadvantages of the iOS operating system-

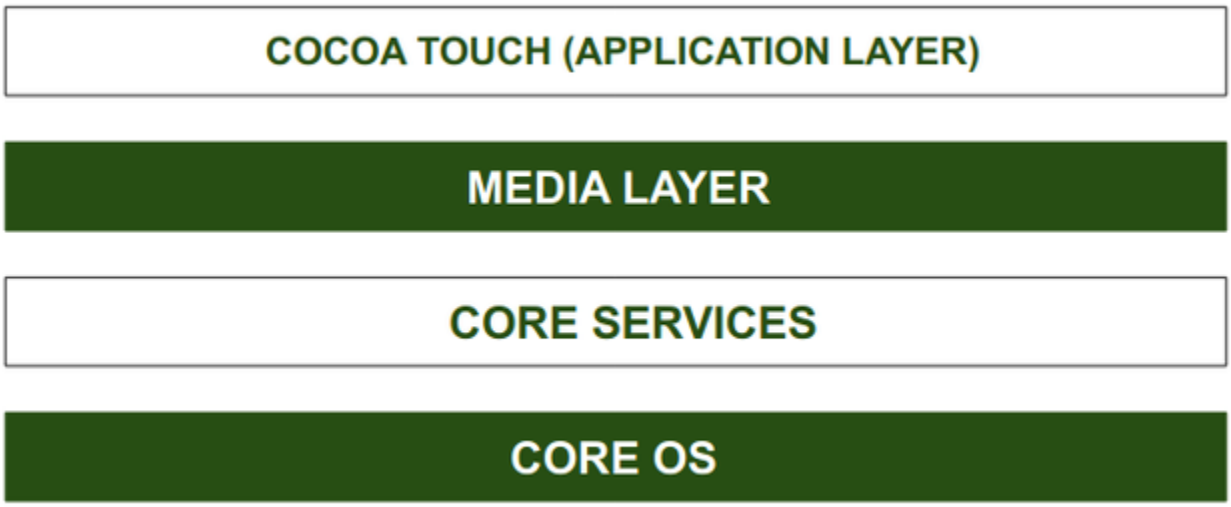
1. More Costly.
2. Less User Friendly as Compared to Android Operating System.
3. Not Flexible as it supports only IOS devices.
4. Battery Performance is poor.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

ARCHITECTURE AND SDK FRAMEWORK

IOS is a Mobile Operating System that was developed by Apple Inc. for iPhones, iPads, and other Apple mobile devices. iOS is the second most popular and most used Mobile Operating System after Android.
The structure of the iOS operating System is Layered based. Its communication doesn't occur directly. The layer's between the Application Layer and the Hardware layer will help for Communication. The lower level gives basic services on which all applications rely and the higher-level layers provide graphics and interface-related services. Most of the system interfaces come with a special package called a framework.

A framework is a directory that holds dynamic shared libraries like .a files, header files, images, and helper apps that support the library. Each layer has a set of frameworks that are helpful for developers.



Architecture of IOS

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

CORE OS Layer:

All the IOS technologies are built under the lowest level layer i.e. Core OS layer. These technologies include:

1. Core Bluetooth Framework
2. External Accessories Framework
3. Accelerate Framework
4. Security Services Framework
5. Local Authorization Framework etc.

It supports 64 bit which enables the application to run faster.

CORE SERVICES Layer:

Some important frameworks are present in the CORE SERVICES Layer which helps the iOS operating system to cure itself and provide better functionality. It is the 2nd lowest layer in the Architecture as shown above. Below are some important frameworks present in this layer:

1. **Address Book Framework-**
   The Address Book Framework provides access to the contact details of the user.
2. **Cloud Kit Framework-**
   This framework provides a medium for moving data between your app and iCloud.
3. **Core Data Framework-**
   This is the technology that is used for managing the data model of a Model View Controller app.
4. **Core Foundation Framework-**
   This framework provides data management and service features for iOS applications.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

5. **Core Location Framework-**
   This framework helps to provide the location and heading information to the application.
6. **Core Motion Framework-**
   All the motion-based data on the device is accessed with the help of the Core Motion Framework.
7. **Foundation Framework-**
   Objective C covering too many of the features found in the Core Foundation framework.
8. **HealthKit Framework-**
   This framework handles the health-related information of the user.
9. **HomeKit Framework-**
   This framework is used for talking with and controlling connected devices with the user's home.
10. **Social Framework-**
    It is simply an interface that will access users' social media accounts.
11. **StoreKit Framework-**
    This framework supports for buying of contents and services from inside iOS apps.

MEDIA Layer:

With the help of the media layer, we will enable all graphics video, and audio technology of the system. This is the second layer in the architecture. The different frameworks of MEDIA layers are:

1. **ULKit Graphics-**
   This framework provides support for designing images and animating the view content.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

2. **Core Graphics Framework-**
   This framework support 2D vector and image-based rendering and it is a native drawing engine for iOS.
3. **Core Animation-**
   This framework helps in optimizing the animation experience of the apps in iOS.
4. **Media Player Framework-**
   This framework provides support for playing the playlist and enables the user to use their iTunes library.
5. **AV Kit-**
   This framework provides various easy-to-use interfaces for video presentation, recording, and playback of audio and video.
6. **Open AL-**
   This framework is an Industry Standard Technology for providing Audio.
7. **Core Images-**
   This framework provides advanced support for motionless images.
8. **GL Kit-**
   This framework manages advanced 2D and 3D rendering by hardware-accelerated interfaces.

COCOA TOUCH:

COCOA Touch is also known as the application layer which acts as an interface for the user to work with the iOS Operating system. It supports touch and motion events and many more features.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

The COCOA TOUCH layer provides the following frameworks :

1. **EvenKit Framework-**
   This framework shows a standard system interface using view controllers for viewing and changing events.
2. **GameKit Framework-**
   This framework provides support for users to share their game-related data online using a Game Center.
3. **MapKit Framework-**
   This framework gives a scrollable map that one can include in your user interface of the app.
4. **PushKit Framework-**
   This framework provides registration support.

FILE SYSTEMS

A Linux file system is a structured collection of files on a disk drive or a partition. A partition is a segment of memory and contains some specific data. In our machine, there can be various partitions of the memory. Generally, every partition contains a file system.

The general-purpose computer system needs to store data systematically so that we can easily access the files in less time. It stores the data on hard disks (HDD) or some equivalent storage type. There may be below reasons for maintaining the file system:

- Primarily the computer saves data to the RAM storage; it may lose the data if it gets turned off. However, there is non-volatile RAM (Flash RAM and SSD) that is available to maintain the data after the power interruption.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

- Data storage is preferred on hard drives as compared to standard RAM as RAM costs more than disk space. The hard disks costs are dropping gradually comparatively the RAM.

The Linux file system contains the following sections:

- The root directory (/)
- A specific data storage format (EXT3, EXT4, BTRFS, XFS and so on)
- A partition or logical volume having a particular file system.

Types of Linux File System

- Linux offers many file systems such as **Ext, Ext2, Ext3, Ext4, JFS, ReiserFS, XFS, btrfs,** and **swap**.

1**. Ext, Ext2, Ext3 and Ext4 file system**

- The file system Ext stands for **Extended File System**. It was primarily developed for **MINIX OS**. The Ext file system is an older version, and is no longer used due to some limitations.
- **Ext2** is the first Linux file system that allows managing two terabytes of data. Ext3 is developed through Ext2; it is an upgraded version of Ext2 and contains backward compatibility. The major drawback of Ext3 is that it does not support servers because this file system does not support file recovery and disk snapshot.
- **Ext4** file system is the faster file system among all the Ext file systems. It is a very compatible option for the SSD (solid-state drive) disks, and it is the default file system in Linux distribution.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

2. JFS File System

- JFS stands for **Journaled File System**, and it is developed by **IBM for AIX Unix**. It is an alternative to the Ext file system. It can also be used in place of Ext4, where stability is needed with few resources. It is a handy file system when CPU power is limited.

3. ReiserFS File System

- ReiserFS is an alternative to the Ext3 file system. It has improved performance and advanced features. In the earlier time, the ReiserFS was used as the default file system in SUSE Linux, but later it has changed some policies, so SUSE returned to Ext3. This file system dynamically supports the file extension, but it has some drawbacks in performance.

4. XFS File System

- XFS file system was considered as high-speed JFS, which is developed for parallel I/O processing. NASA still using this file system with its high storage server (300+ Terabyte server).

5. Btrfs File System

- Btrfs stands for the **B tree file system**. It is used for fault tolerance, repair system, fun administration, extensive storage configuration, and more. It is not a good suit for the production system.

6. Swap File System

- The swap file system is used for memory paging in Linux operating system during the system hibernation. A system that never goes in

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

hibernate state is required to have swap space equal to its RAM size.

Linux File System Structure

A file system mainly consists of 3 layers. From top to bottom:

1. Logical File System:

The Logical File System acts as the interface between the user applications and the file system itself. It facilitates essential operations such as opening, reading, and closing files. Essentially, it serves as the user-friendly front-end, ensuring that applications can interact with the file system in a way that aligns with user expectations.

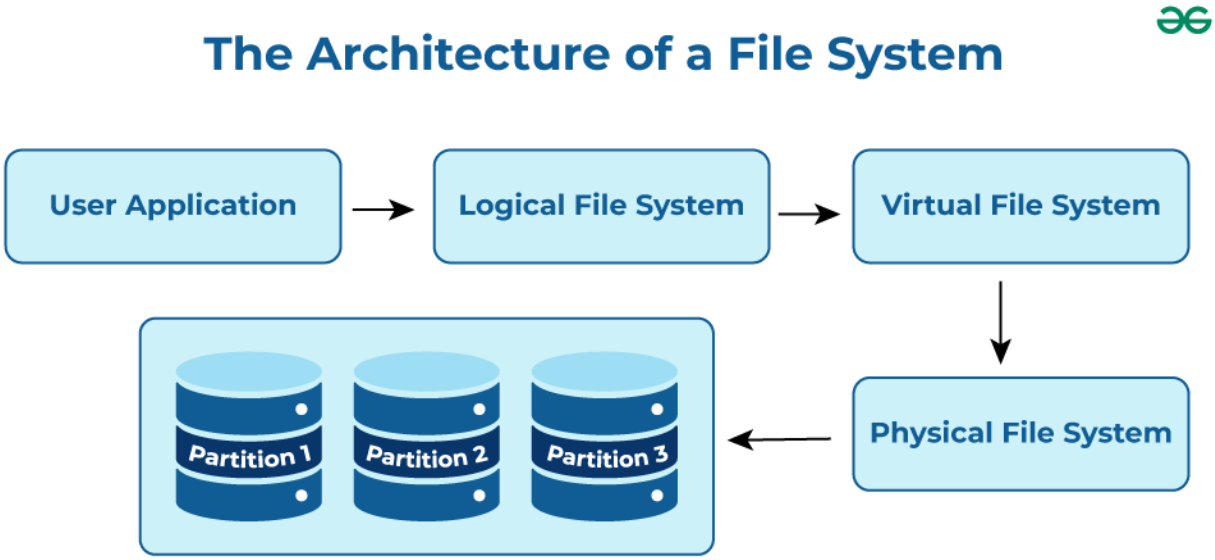
2. Virtual File System:

The Virtual File System (VFS) is a crucial layer that enables the concurrent operation of multiple instances of physical file systems. It provides a standardized interface, allowing different file systems to coexist and operate simultaneously. This layer abstracts the underlying complexities, ensuring compatibility and cohesion between various file system implementations.

3. Physical File System:

The Physical File System is responsible for the tangible management and storage of physical memory blocks on the disk. It handles the low-level details of storing and retrieving data, interacting directly with the hardware components. This layer ensures the efficient allocation and utilization of physical storage resources, contributing to the overall performance and reliability of the file system.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)



ext4 in Linux File System

Ext4 was designed to be backward compatible with ext3 and ext2, its previous generations. It's better than the previous generations in the following ways:

- It provides a **large file system** as described in the table above.
- Utilizes **extents** that improve large file performance and reduces fragmentation.
- Provides **persistent pre-allocation** which guarantees space allocation and contiguous memory.
- **Delayed allocation** improves performance and reduces fragmentation by effectively allocating larger amounts of data at a time.
- It uses HTree indices to allow **unlimited number of subdirectories.**
- Performs **journal checksumming** which allows the file system to realize that some of its entries are invalid or out of order after a crash.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

- Support for time-of-creation timestamps and **improved timestamps** to induce granularity.
- Transparent encryption.
- Allows cleaning of inode tables in background which in turn speeds initialization. The process is called **lazy initialization**.
- Enables **writing barriers** by default. Which ensures that file system metadata is correctly written and ordered on disk, even when write caches lose power.

There are still some features in the process of developing like metadata checksumming, first-class quota supports, and large allocation blocks.

However, ext4 has some limitations. Ext4 does not guarantee the integrity of your data, if the data is corrupted while already on disk then it has no way of detecting or repairing such corruption. The ext4 file system cannot secure deletion of files, which is supposed to cause overwriting of files upon deletion. It results in sensitive data ending up in the file-system journal.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)